

Linear Programming of Graph Squares

Eddie Xu

Submitted for the degree of Master of Science:
Mathematics and Statistics

Supervisor: Christopher Duffy

August 26, 2026

Abstract

The square of a graph is obtained by joining every pair of vertices whose distance is at most two. Given a graph G , the graph square root problem asks whether there exists a graph H such that $G = H^2$ and, if so, whether such a root can be constructed in polynomial time. Whilst this problem admits polynomial-time algorithms for restricted classes of root graphs, the general decision problem is NP-complete.

In this thesis, rather than studying the construction problem directly and utilizing reduction techniques to ascertain polynomial-time algorithms, we formulate the problem as an integer program based on the relationship between the adjacency matrices of a root H and its square G . By constructing the corresponding linear relaxation, the problem can then be situated within polynomial time through the use of interior-point methods.

The seminal result for the problem is the work by Ross and Harary, who approached the problem by restricting the root graphs to trees. By analyzing the number of cliques which each vertex in the graph square is situated in, a polynomial-time algorithm was given for finding the unique tree root up to isomorphism. In contrast, our formulation provides an optimization-based approach that can be iteratively strengthened through the addition of further constraints. These constraints progressively refine the linear relaxation, enabling tree roots to be represented by integral solutions, while graphs with no possible roots are identified through fractional or infeasible solutions. Rather than relying on heuristic such as rounding, our approach exploits structural properties of the graph square and its root to obtain a deterministic solution.

Contents

1	Introduction	3
1.1	Background	3
1.2	Literature	4
1.2.1	Overview	5
1.3	Definitions and Notations	6
1.3.1	Complexity Theory	6
1.3.2	Graph Theory	7
1.3.3	Mathematical Optimization	11
2	Algorithmic Approaches	13
2.1	NP-Completeness Proof	13
2.2	Polynomial Time Algorithm	13
2.3	NP-Completeness Proofs	15
2.4	Integer Program Foundation	15
2.4.1	Integer Program Formulation	18
2.4.2	Validity	18
2.5	Linear Program Relaxation	21
2.5.1	Formulation	21
2.6	Max Linear Program	22
2.6.1	Formulation	22
2.6.2	Fundamental Properties	22
3	Square of A Tree using Linear Programming	24
3.1	Single Triangular Decomposition	24
3.1.1	Formulation	24
3.1.2	Validity	25
3.2	Tetrahedron Decomposition	26
3.2.1	Formulation	26
3.2.2	Validity	26
3.3	Tree Proof	28
3.3.1	Integer Program	28
3.3.2	Linear Program	28
3.3.3	Max Linear Program	28
4	Further Work	30
4.1	Symbolic Groups	30
4.2	Duality	31

4.2.1	Rounding	34
-------	--------------------	----

Chapter 1

Introduction

1.1 Background

In 1949, Leon Festinger proposed a novel method in the field of social psychology for analyzing relationships among a number of people present [FBB23]. Classically, this was represented using a sociogram, a visual tool that maps out the relationships between people. However, Forsyth and Katz introduced the use of a matrix to represent the sociometric pattern, with rows and columns corresponding to individuals and containing 1s for choices and blank diagonals. Festinger noticed that his structure, reminiscent of the adjacency matrix defined by König in [Kön36], also had the remarkable property that the action of squaring a matrix

$$A_{rc}^2 = A_{1c}A_{r1} + \cdots + A_{nc}A_{rn}^1$$

represented the number of “two-step” connections, which can then be used to denote “three-step” connections between any two people

$$A_{rc}^3 = A_{1c}A_{r1}^2 + \cdots + A_{nc}A_{rn}^2,^2$$

such that higher steps can be easily generated. In this manner, higher powers of the adjacency matrix encode the number of walks of increasing length between pairs of individuals, allowing densely connected groups of people, known as cliques, to be identified.

In the process of improving Festinger’s clique detection methods, Ross and Harary showed that complete graphs may be realized as squares of star trees by considering powers of such matrices [RH60]. They introduced a structural characterization of the square of a graph in terms of clique intersections, “multicliquical” vertices, and so called “neighborly” properties, proving that a graph is the square of a tree if and only if

- i) every vertex is “neighborly” - is not isolated and has an edge to another vertex,
- ii) no two cliques intersect in more than two vertices, and

¹ A_{rc}^2 is more naturally written as $\sum_{k=1}^n A_{ck}A_{kr}$ using the commutativity of multiplication $A_{1c}A_{r1} = A_{c1}A_{1r}$, which intuitively gives the notion of paths from c to r via intermediary k

² A_{rc}^3 is more naturally written as $\sum_{k=1}^n A_{ck}A_{kr}^2$, which intuitively counts the number of paths from c to r that go through an intermediate vertex k , followed by any two-step path from k to r .

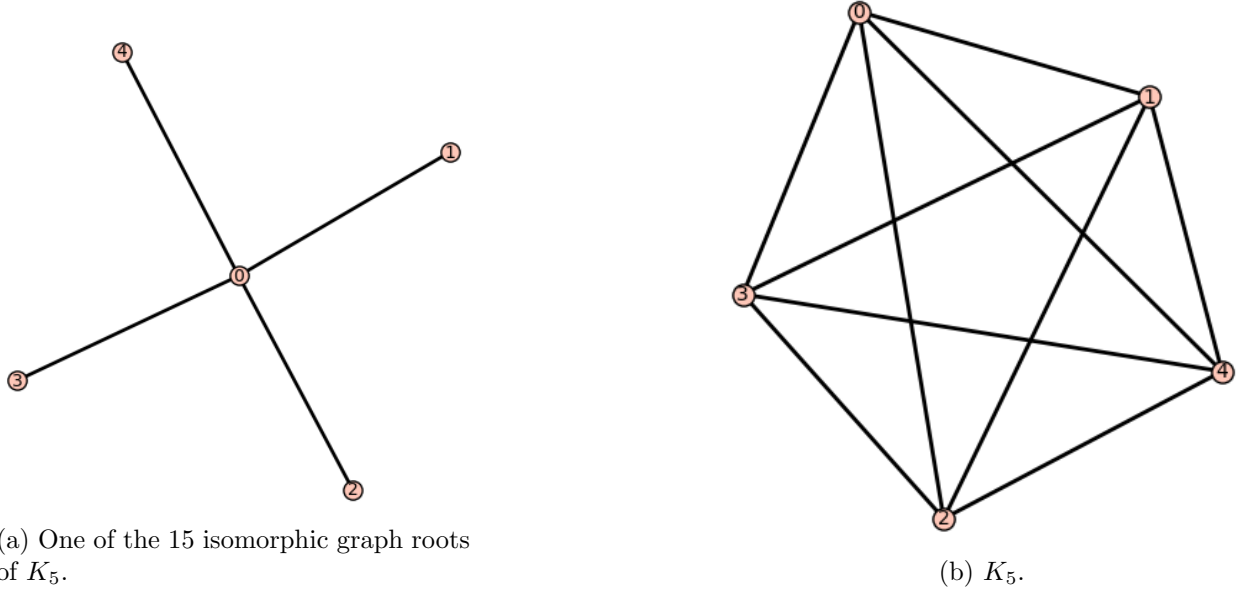


Figure 1.1: The star graph $K_{1,4}$ and its square K_5 , which is a complete graph since every vertex in the root .

- iii) the number of clique pairs intersecting in two vertices is exactly one less than the total number of cliques.

Using these properties, they constructed an explicit algorithm that recovers the underlying tree for a given graph square. Their seminal paper has become the foundational reference for the study of graph squares and is universally cited in subsequent literature on the subject, and will be explored in more detail later.

Since then, the problem of finding the square root of a graph has extended beyond sociology and is a well-studied field in both mathematics and computer-science, with direct applications in distributed computing (e.g., see [PA23]), network design and computational biology. This is in part because many structural properties of graph roots are first established in the square case before being extended to the more general k -th power setting (e.g., see [HKT67]).

Research in this area has primarily focused around an undirected graph H and its square $G = H^2$ which is obtained by adding an edge between every pair of vertices whose distance in H is at most two. This two-step undirected graph square has been studied extensively with much of the focus being on the complexity of the decision problem - given a specific graph square G does there exist a root graph H such that $H^2 = G$. Motwani and Sudan showed that graph squares are NP-complete in general and hence subsequent studies in the area have restricted the decision problem in two ways, either by considering various properties on the root graph or by considering properties on the graph square.

1.2 Literature

Focusing first on special undirected graph classes, the aforementioned work by Ross and Harary in 1960 gives an algorithm for constructing a tree root by using characterizations of the graph square. As part of this work they proved tree square roots are unique up to isomorphism. This was the first major contribution to the study of graph roots and provided fundamental structural

characteristics of graph squares, laying the groundwork for later research on graph root problems and their applications [RH60].

Fleischer in 1974 showed that as long as a graph is 2-connected its graph square will have a Hamiltonian cycle. This established a broad condition guaranteeing Hamiltonicity in graph squares which is used in many subsequent works [Fle74].

Lau and Corneil in 2004 provided an algorithm for constructing square roots from an interval square in polynomial time, and showed that the decision problem for finding roots of chordal graphs, roots of split graphs, and chordal roots of graphs are NP-complete [LC04]. Furthermore, Lau considered the construction of square roots from a bipartite graph in linear time, and showed that the existence of cube roots in bipartite square is NP-complete [Lau06].

Le and Tuy in 2007 generalized tree-root results further by giving a linear time recognition of roots of block graphs [LT07].

Extending to the broader generalizing of graph squares, Mukhopadhyay in 1967 provided necessary and sufficient conditions for a connected graph to have a generalized square root [Muk67], whilst Geller in 1970 likewise found necessary and sufficient conditions for a digraph to have a square root [Gel70].

In 1994 Motwani and Sudan established the complexity of the square problem for the generalized undirected case and showed that the problem of determining if a given graph has a square root is a NP-complete problem via a polynomial reduction to 3-CNF Not-All-Equal SAT [MS94].

Their work led to joint work by Farzad, Le, Tuy, Lau and Karimi in 2009 and 2012, in which they established a partial dichotomy theorem for the complexity of determining if a given undirected graph of girth n has a square root [Far+09] which was informally completed under [FK12]. In 2013, Cochefert and others gave an algorithm for constructing a square root with minimal number of edges from undirected graphs with max degree ≤ 6 in polynomial time and an algorithm for constructing maximal edge square roots in exponential time [Coc+13].

In recent years, Duffy provided a full classification for the undirected square of oriented graphs satisfying max degree ≤ 3 and girth ≥ 4 [Duf25].

1.2.1 Overview

In addition to providing a general overview of the research direction in this area we first standardize the definitions that will be used for the rest of this thesis. We then introduce fundamental theory on the three areas of complexity theory, graph theory and mathematical optimization that are used in the various algorithmic approaches to the decision problem.

The first algorithm we explore is the proof used by Motwani and Sudan in showing that the decision problem determining whether a given graph G admits a square root is NP-complete in general [MS94]. We then introduce integer programming, and show that our formulation satisfies correctness and validity. Next, we introduce the polynomial time algorithm given by Ross and Harary for constructing a tree root from a given square in detail.

In Chapter 3 we present the ideas of linear relaxation and discuss the results of each program and their classification into integral, fractional or infeasible solutions.

Finally, we introduce additional augmentations that can be added to the linear program, the structures that bound both the square and the root, and the various methods that can be used to improve the algorithm.

1.3 Definitions and Notations

1.3.1 Complexity Theory

Complexity theory studies the classification of computational problems and explores the amount of resources required to solve certain problems. The critical resource that is measured is running time and is standardized across various different problems by using

Definition 1.3.1 (Big- O Notation). *Let $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. The **Big- O Notation** $f(n) = O(g(n))$ is written if there exist constants $c > 0$ and $n_0 \in \mathbb{N}$ such that for all $n \geq n_0$*

$$f(n) \leq cg(n).$$

Using this notation, we can classify algorithms based on their runtime.

Definition 1.3.2 (Polynomial-Time Algorithm). *An algorithm is said to run in **polynomial time** if there exists a constant $c \geq 0$ such that its run time on inputs of size n is $O(n^c)$.*

Polynomial-time algorithms are important because they are generally regarded as efficient and computationally “simple”. Consequently, the class of problems that admit polynomial-time algorithms forms the basis for the complexity class **P**.

Definition 1.3.3 (Decision Problem). *A **decision problem** is a computational problem whose output is either **Yes** or **No** for every valid input instance.*

Definition 1.3.4 (Class **P**). *The complexity class **P** consists of all decision problems that can be solved by a deterministic algorithm in polynomial time.*

Unfortunately many important decision problems are not known to admit such polynomial-time algorithms and hence need additional classification on their “hardness”. For these problems, despite not have an efficient algorithm for finding a solution, a proposed solution can be verified efficiently.

Definition 1.3.5 (Certificate). *Let L be a decision problem. A **certificate** for an instance x of L is a finite string c that provides evidence that x is a **Yes-instance** of L . A certificate is said to be **polynomially verifiable** if there exists a deterministic algorithm that for given input (x, c) , can determine whether c is a valid certificate for x in time polynomial in $|x|$.*

Definition 1.3.6 (Class **NP**). *The complexity class **NP** consists of all decision problems for which every **Yes-instance** admits a certificate that is polynomially verifiable.*

Since every decision problem in **P** can be solved in polynomial time, a solution can also be verified in polynomial time,

$$\mathbf{P} \subseteq \mathbf{NP}.$$

Whether this inclusion is strict remains one of the most important open questions in theoretical computer science - the **P** vs **NP** problem. As **NP** is a large class of decision problems, the relative difficulty of problems in **NP** is compared by using reduction.

Definition 1.3.7 (Polynomial-Time Reduction). *Let A and B be decision problems. A **polynomial-time reduction** from A to B , denoted $A \leq_p B$, is a polynomial-time computable function f such that, for every instance x of A ,*

$$x \in A \iff f(x) \in B.$$

This notion of reduction allows the computational difficulty of problems to be compared. In particular, a problem is **NP-hard** if every problem in **NP** can be reduced to it in polynomial time.

Definition 1.3.8 (Class **NP-Hard**). *A decision problem B is **NP-hard** if every problem $A \in \mathbf{NP}$ satisfies $A \leq_p B$. The collection of all **NP-hard** problems is denoted by the class **NP-hard**.*

Importantly, an **NP-hard** problem may not necessarily lie in **NP**. If it does, it is classified as **NP-complete**.

Definition 1.3.9 (Class **NP-Complete**). *A decision problem B is **NP-complete** if $B \in \mathbf{NP}$, and B is **NP-hard**. The collection of all **NP-complete** problems is denoted by the class **NP-complete**.*

That is, one can establish that a problem is **NP-complete** by showing that it belongs to **NP** and that a known **NP-complete** problem can be reduced to it in polynomial time.

The first known **NP-complete** problem was established by the Cook-Levin theorem and is the Boolean satisfiability problem, or SAT.

Definition 1.3.10 (Boolean Satisfiability Problem). *The **Boolean satisfiability problem (SAT)** asks whether there exists an assignment of truth values to the variables of a given Boolean formula such that the formula evaluates to true. A Boolean formula is said to be **satisfiable** if such an assignment exists.*

Any Boolean formula can be transformed into an equisatisfiable formula typically in linear time using the **Tseytin transformation** by introducing additional variables. This transformation preserves satisfiability while ensuring only linear growth in the size of the resulting conjunctive normal form Boolean formula.

Definition 1.3.11 (Conjunctive Normal Form). *A Boolean formula is in **conjunctive normal form (CNF)** if it is expressed as a conjunction of clauses, where each clause is a disjunction of literals. That is, a CNF formula has the form*

$$C_1 \wedge C_2 \wedge \cdots \wedge C_m, \quad \text{where each clause } C_i \text{ is of the form } C_i = \ell_{i1} \vee \ell_{i2} \vee \cdots \vee \ell_{ik_i}$$

and each ℓ_{ij} is either a Boolean variable or its negation denoted with $\neg$.

A further restriction is obtained by requiring each clause in a CNF formula to contain exactly three literals, which can be done using 3-CNF transformation, to give the following problem.

Definition 1.3.12 (3-SAT). *The **3-SAT** problem asks whether there exists an assignment of truth values to the variables of a Boolean formula in conjunctive normal form such that every clause contains exactly three literals and the formula evaluates to true.*

3-SAT is a **NP-complete** problem and will be used as the source problem for establishing the **NP-hardness** of the graph square problem.

1.3.2 Graph Theory

Objects, which are represented by vertices and edges representing relationships between them can be conveniently represented using a

Definition 1.3.13 (Graph). *A **graph** G is an ordered pair (V, E) , where V is a set of vertices and E is a set of edges between pairs of vertices in V .*

The number of vertices and edges in a graph provide basic measures of its size.

Definition 1.3.14 (Order and Size). The **order** of a graph G , denoted by $|G|$, is the number of vertices in G , while the **size** of G is the number of edges. Thus, if $V(G) = \{v_1, \dots, v_n\}$ and $E(G) = \{e_1, \dots, e_m\}$, then $|G| = n$ and $|E(G)| = m$.

For a vertex v , the number of edges incident to it is its degree.

Definition 1.3.15 (Degree). The **degree** of a vertex v in a graph G , denoted by $\deg_G(v)$, is the number of edges incident to v . The minimum and maximum degrees of G are denoted by

$$\delta(G) = \min_{v \in V(G)} \deg_G(v) \quad \text{and} \quad \Delta(G) = \max_{v \in V(G)} \deg_G(v),$$

respectively.

The degree of a vertex can also be described through the vertices to which it is adjacent to.

Definition 1.3.16 (Neighbourhood). For a vertex $v \in V(G)$, the **neighbourhood** of v denoted by $N_G(v)$, is the set of vertices adjacent to v in G . The **closed neighbourhood** of v includes vertex v and is denoted by

$$N_G[v] = N_G(v) \cup \{v\}.$$

Connections between vertices can be described using walks, paths and cycles.

Definition 1.3.17 (Walk). A **walk** from a vertex v_i to a vertex v_j in a graph G is an alternating sequence of vertices and edges

$$W = \langle v_i, e_1, v_1, e_2, \dots, e_k, v_j \rangle,$$

where each edge e_r has endpoints given by the vertices immediately preceding and following it in the sequence. The number of edges in W is called the **length** of the walk.

Definition 1.3.18 (Path). A **path** is a walk in which no vertex is repeated.

Definition 1.3.19 (Cycle). A **cycle** is a closed path in which the first and last vertices are the same and all other vertices are distinct.

The length of a shortest cycle is an important structural parameter.

Definition 1.3.20 (Girth). The **girth** of a graph G , denoted by $g(G)$, is the length of the shortest cycle in G . If G contains no cycle, its girth is ∞ .

The length of a shortest path provides a natural measure of the distance between two vertices.

Definition 1.3.21 (Distance). The **distance** between two vertices u and v on the graph G , denoted by $d_G(u, v)$, is the number of edges of the shortest path between vertices u and v in G . If no such path exists, $d_G(u, v) = \infty$.

Definition 1.3.22 (Diameter). The **diameter** of a graph G is

$$\text{diam}(G) = \max_{u, v \in V(G)} d_G(u, v).$$

For a disconnected graph, the diameter is ∞ .

If a graph has finite diameter it is connected.

Definition 1.3.23 (Connected Graph). A graph G is **connected** if there exists a path between every pair of vertices in G . Otherwise, G is **disconnected**.

A graph can be further categorized according to whether multiple edges between a vertex and itself are permitted, whether there are infinite number of vertices and edges, whether the edges represent an unordered or ordered relationship between their endpoints, and whether the vertices are labelled or unlabelled.

Definition 1.3.24 (Simple Graph). A **simple graph** $G = (V, E)$ is a graph in which no two distinct edges connect the same pair of vertices and no edge connects a vertex to itself.

Definition 1.3.25 (Finite Graph). A **finite graph** is a graph $G = (V, E)$ for which both V and E are finite sets. A graph that is not finite is called an **infinite graph**.

Definition 1.3.26 (Directed Graph). A **directed graph** $G = (V, E)$ is a graph in which each edge $e \in E$ is an ordered pair of distinct vertices in V . Thus, (u, v) represents an edge directed from u to v , where (u, v) may not necessarily be equal to (v, u) .

An **undirected graph** $G = (V, E)$ is a graph in which each edge $e \in E$ is an unordered pair of distinct vertices in V . Thus, $\{u, v\} = \{v, u\}$ for every edge between vertices u and v .

Definition 1.3.27 (Labelled Graph). A **labelled graph** is a graph $G = (V, E)$ together with a labelling function $\ell : V \rightarrow L$, where L is a set of labels. A graph is **unlabelled** when no such vertex labels are specified.

In this thesis, we exclusively study finite, simple, undirected, unlabelled graphs. However, the constructions and algorithms may use additional structure, including labels or multiple edges, which must be analysed and restricted to their corresponding simple, unlabelled counterparts.

In particular, two graphs may have different vertex sets while nevertheless having the same underlying structure. This motivates the following notion of isomorphism.

Definition 1.3.28 (Graph Isomorphism). Let G and H be graphs. An **isomorphism** from G to H is a bijection

$$f : V(G) \rightarrow V(H) \quad \text{such that} \quad uv \in E(G) \text{ if and only if } f(u)f(v) \in E(H).$$

We say that G is **isomorphic** to H , written $G \cong H$, if there exists an isomorphism from G to H .

The isomorphism relation is an equivalence relation on the set of graphs, and an **isomorphism class** is an equivalence class of graphs under this relation. Thus, graphs within the same isomorphism class have the same structure up to a relabelling of their vertices.

We will also consider graphs obtained by restricting the vertices and edges of another graph.

Definition 1.3.29 (Subgraph). A graph $H = (V(H), E(H))$ is a **subgraph** of a graph $G = (V(G), E(G))$ if

$$V(H) \subseteq V(G) \quad \text{and} \quad E(H) \subseteq E(G).$$

For $S \subseteq V(G)$, the **induced subgraph** of G on S , denoted by $G[S]$, is the graph with vertex set S and edge set

$$E(G[S]) = \{uv \in E(G) : u, v \in S\}.$$

One of the most important types of graphs is the complete graph, and its subgraph counterpart.

Definition 1.3.30 (Complete Graph). A graph on n vertices is **complete** if every pair of distinct vertices is adjacent. The complete graph on n vertices is denoted by K_n .

Definition 1.3.31 (Clique). A **clique** of a graph G is a subset $S \subseteq V(G)$ such that the induced subgraph $G[S]$ is complete. The **clique number** of G , denoted by $\omega(G)$, is the size of a largest clique in G .

The graph structures above can be represented algebraically using matrices.

Definition 1.3.32 (Adjacency Matrix). The **adjacency matrix** of graph H is denoted by $A_H \in \{0, 1\}^{n \times n}$ where

$$(A_H)_{ij} = \begin{cases} 1, & \text{if } ij \in E_H, \\ 0, & \text{otherwise,} \end{cases}$$

The degrees of the vertices can similarly be represented by a diagonal matrix.

Definition 1.3.33 (Degree Matrix). The **degree matrix** of graph H is denoted by $D_H \in \mathbb{R}^{n \times n}$ where

$$(D_H)_{ij} = \begin{cases} \deg_H(i), & \text{if } i = j, \\ 0, & \text{otherwise.} \end{cases}$$

Definition 1.3.34 (Indicator Function). Let $\mathbb{1}_{\geq 1} : \mathbb{R}^{n \times n} \rightarrow \{0, 1\}^{n \times n}$ denote the entrywise **indicator function**, defined by

$$\mathbb{1}_{\geq 1}(M)_{ij} = \begin{cases} 1, & \text{if } M_{ij} \geq 1, \\ 0, & \text{otherwise.} \end{cases}$$

We now introduce the graph-theoretic objects central to this thesis.

Definition 1.3.35 (Graph Root). Let the **graph root** $H = (V, E_H)$ be a finite, simple, undirected graph, where V is the vertex set with $|V| = n$, and $E_H \subseteq \{\{u, v\} \mid u, v \in V, u \neq v\}$ is the edge set.

Definition 1.3.36 (Square of a Graph). The **square of a graph** H , denoted by $H^2 = (V, E_{H^2})$ is the graph on the same vertex set with edge set

$$E_{H^2} = \{uv \mid u \neq v, 1 \leq d_H(u, v) \leq 2\}. \quad (1.1)$$

That is, two distinct vertices are adjacent in H^2 if and only if they are connected by a path of length one or two in H . This is the standard definition of the square of a graph.

Definition 1.3.37 (Graph Square). The **graph square** of H is the undirected graph defined on the same vertex set $G = (V, E_G)$. Define the adjacency matrix of G using the adjacency and degree matrices of H

$$A_G = \mathbb{1}_{\geq 1}((A_H)^2 - D_H + A_H). \quad (1.2)$$

Definition 1.3.38. A pendant subgraph is the induced subgraph on three vertices s, u, v where s is a leaf with unique neighbour u , and u is a stem of degree 2 with neighbours s and v , where v connects the subgraph to the remainder of the tree.

Definition 1.3.39. A leaf clique subgraph is the induced subgraph on four vertices s_1, s_2, u, v where s_1, s_2 are leaves with unique neighbour u , and u is an internal vertex of degree ≥ 3 where $\{s_1, s_2, v\} \subseteq N(u)$, where v connects the subgraph to the remainder of the tree.

1.3.3 Mathematical Optimization

Mathematical optimization studies the problem of finding the best solution from all feasible solutions which are defined by a collection of constraints. An optimization problem usually consists of decision variables, an objective function to be maximized or minimized, and a set of constraints restricting the feasible values of the variables.

Definition 1.3.40 (Integer Program). An **integer program** (IP) is an optimization problem of the form

$$\begin{aligned} \max/\min \quad & c^\top x \\ \text{subject to} \quad & Ax \leq b, \\ & x \in \mathbb{Z}^n, \end{aligned}$$

where $x \in \mathbb{Z}^n$ is the vector of decision variables, $c \in \mathbb{R}^n$ is the objective coefficient vector, $A \in \mathbb{R}^{m \times n}$ is the constraint matrix, and $b \in \mathbb{R}^m$ is the constraint vector.

A linear relaxation is obtained by removing the integrality requirements from an integer program while retaining its linear objective function and constraints.

Definition 1.3.41 (Linear Relaxation). Let IP be an integer program. The **linear relaxation** of IP is the linear program obtained by removing the requirement that its variables take integer values.

Definition 1.3.42 (Linear Program). A **linear program** (LP) is an optimization problem of the form

$$\begin{aligned} \max/\min \quad & c^\top x \\ \text{subject to} \quad & Ax \leq b, \\ & x \in \mathbb{R}^n, \end{aligned}$$

where $x \in \mathbb{R}^n$ is the vector of decision variables, $c \in \mathbb{R}^n$ is the objective coefficient vector, $A \in \mathbb{R}^{m \times n}$ is the constraint matrix, and $b \in \mathbb{R}^m$ is the constraint vector.

Definition 1.3.43 (Feasible Solution). A **feasible solution** to an optimization problem is an assignment of values to its decision variables that satisfies all of its constraints. The set of all feasible solutions is called the **feasible set** and is denoted by $\mathcal{F}$. Geometrically, $\mathcal{F}$ is the subset of the decision-variable space satisfying all of the constraints and is referred to as the **feasible region**.

Among the feasible solutions, an optimal solution is one that achieves the best value of the objective function.

Definition 1.3.44 (Optimal Solution). An **optimal solution** is a feasible solution that maximizes or minimizes the objective function, depending on the optimization problem. The corresponding value of the objective function is called the **optimal value** and is denoted by OPT.

When an integer program is relaxed to a linear program, every feasible integer solution remains feasible for the relaxation. Consequently, if $\mathcal{F}_{IP}$ and $\mathcal{F}_{LP}$ denote the feasible sets of the integer program and its linear relaxation respectively, then

$$\mathcal{F}_{IP} \subseteq \mathcal{F}_{LP}.$$

Geometrically, $\mathcal{F}_{IP}$ is represented by the feasible integer points in the decision-variable space, whereas $\mathcal{F}_{LP}$ is the polyhedron defined by the relaxed constraints and may contain additional points.

For a maximization problem, this set inclusion implies $\text{OPT}_{IP} \leq \text{OPT}_{LP}$, so the linear relaxation provides an upper bound on the optimal value of the integer program. For a minimization problem, $\text{OPT}_{LP} \leq \text{OPT}_{IP}$, so the linear relaxation provides a lower bound.

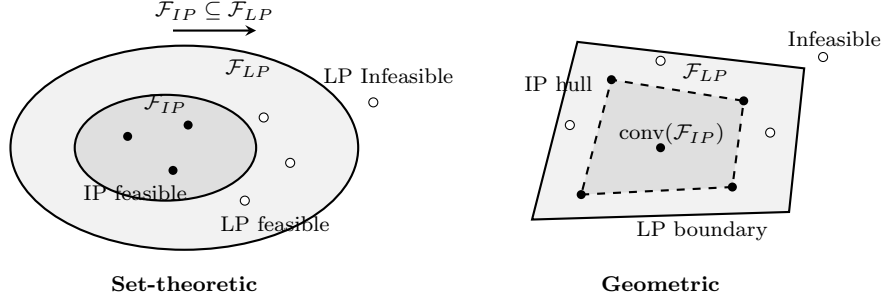


Figure 1.2: Set-theoretic and geometric relationships between an integer program and its linear relaxation. Solutions that are IP feasible, LP feasible may be

Definition 1.3.45 (Integer Hull). *Let $\mathcal{F}_{IP}$ denote the feasible set of an integer program. The **integer hull** is the convex hull of the feasible integer solutions, denoted by $\text{conv}(\mathcal{F}_{IP})$. That is, the integer hull is the set of all convex combinations of points in $\mathcal{F}_{IP}$.*

Since the feasible region of a linear program is convex, the integer hull is contained in the feasible region of its linear relaxation. Hence,

$$\mathcal{F}_{IP} \subseteq \text{conv}(\mathcal{F}_{IP}) \subseteq \mathcal{F}_{LP}.$$

The integer hull is the smallest convex set containing all feasible integer solutions. Thus, geometrically, the gap between the integer hull and the LP feasible region represents the additional solutions admitted by the relaxation. If

$$\text{conv}(\mathcal{F}_{IP}) = \mathcal{F}_{LP},$$

then the linear relaxation coincides with the integer hull and is therefore an exact convex description of the feasible integer solutions.

Chapter 2

Algorithmic Approaches

In this chapter we explore different methodologies that have been used to show certain properties of the graph root and the graph square. For many restricted cases which are **NP**, a polynomial time reduction is given and in cases which are polynomial, a polynomial time algorithm is given.

2.1 NP-Completeness Proof

We now establish the computational complexity of the graph square problem as was established by Motwani and Sudan in their paper [MS94]. The decision version of the problem asks whether a given graph admits a graph root whose square is the given graph.

Definition 2.1.1 (SQUARE).

***Instance:** A finite, simple, undirected graph $G = (V, E)$.*

***Question:** Does there exist a finite, simple, undirected graph H such that*

$$G = H^2?$$

To show that Definition 2.1.1 is **NP**-complete, it is necessary to show that it is in **NP** and that it is **NP**-hard by reducing not-all-equal SAT (NAESAT) to the problem.

Given a candidate square root H , one can verify in polynomial time whether $G = H^2$ by computing the distances between pairs of vertices in H , or equivalently by constructing the square H^2 and comparing its edge set with that of G . Hence, H provides a polynomially verifiable certificate for a **Yes**-instance of SQUARE and therefore

$$\text{SQUARE} \in \text{NP}.$$

2.2 Polynomial Time Algorithm

Ross and Harary give an algorithm for constructing a tree T when given its graph square $G = T^2$ using the following theorem

Theorem 1 (Structural Characterization of Squares of Trees). *Let G be a finite, simple, undirected graph. Then $G = T^2$ for some tree T if and only if either $G = K_p$ is complete, or the maximal cliques of G satisfy the following properties:*

- (i) *Every vertex of G forms a clique with its neighbours and G is connected*

- (ii) If two cliques intersect at only one point v then there is a third clique with which they share v and exactly one other point
- (iii) A clique $C(v)$ in G corresponding to v in T contains exactly as many points in multiple cliques as the number of cliques which include v
- (iv) Every pair of cliques intersect at at most two points
- (v) The number of pairs of cliques that meet at two points is one less than the number of cliques

To show this, we require the following lemmas

Lemma 1. *G is complete if and only if T has depth one.*

Proof. Let T be a tree with depth one, hence all vertices in T are at most distance two away from any other vertex since the root is a non-leaf vertex adjacent to all vertices in T implies that $G = T^2$ is complete since edges exist between all vertices

Let $G = T^2$ be complete, then all vertices must have distance at most two in T and T has depth at most two. Since T is a tree, there are no cycles and hence there cannot be depth two since this would lead to a path of length three from the vertex at depth two to an alternate vertex in depth one. Hence T has exactly depth one. $\square$

Lemma 2. *Every vertex of G forms a clique with its neighbours and G is connected*

Proof. For a vertex $v \in T$, its neighbours in T are all adjacent edge in G by construction hence $\{v\} \cup \Gamma(v)$ is a clique in G . $T \subseteq G$ and since T is a tree which is connected, G must also be connected. $\square$

Lemma 3. *Every maximal clique of G corresponds to a closed neighbourhood of a non-leaf vertex and every closed neighbourhood of a non-leaf vertex in T form a clique in G*

Proof. A set of leaves in T cannot form a clique in G without having at least one non-leaf since the distance between every leaf is at least greater than two unless they share a common neighbour.

Consider a non-leaf vertex $v \in T$, since all vertices in neighbourhood $\Gamma(v)$ have distance at most two in G , they will form a clique in G . This clique is maximal since all neighbours of v are already part of the clique and no additional vertex can be added. $\square$

Lemma 4. *If G has more than one maximal clique, then vertex v is in exactly one maximal clique in G if and only if v is a leaf in T*

Proof. Suppose vertices v, w to be in two cliques, there must be a neighbour $x \in \Gamma(v)$ that is not adjacent to w . If x is not a leaf in T , will form a third clique in G and being distance two away from w will also be in the clique formed from w . Hence only if this adjacent vertex is a leaf will it be in only one clique G .

Suppose v is a leaf in T whereby v has exactly one neighbour w with is a non-leaf. The only maximal clique in G that includes v must also include w and the neighbourhood $\Gamma(w)$. Since v has no other neighbours it cannot appear in any other clique and hence is in exactly one clique. $\square$

Lemma 5. *Two non-leaves of T are adjacent in T if and only if their neighbourhoods are cliques of G that intersect in exactly two points*

Proof. Supposes cliques in G correspond to the neighbourhoods of two non-leaf neighbours v, w which intersect at exactly v, w . Then v, w must be adjacent, otherwise if they were instead distance two away then they would share a common neighbour u which is also in both cliques.

Let v, w be adjacent vertices that are non-leaves in T . Both will form cliques in G and be in each clique since they are neighbours. Assume to the contrary that $\exists$ vertex k also in both cliques. k must be adjacent to both v, w in T but this creates a cycle since there is now two different paths between v, w . Hence v, w are the only two vertices that intersect in the two clique neighbourhoods. $\square$

Lemma 6. G is a connected graph where the removal of any edge does not result in a disconnected graph

Proof. By construction $\exists$ two paths between every pair of vertices $v, w \in G$ where one of the vertices v must be non-leaf. Remove the existing vw edge in $T \subset G$. There still exists a path by taking the distance two edge to the neighbour u of v and then going from $utow$ (vu, uw), hence the graph G is still connected. $\square$

Combining this gives the following proof of Theorem (??)

Proof. $\square$

Theorem (??) gives the following algorithm. Given an inputted tree square, it returns the tree root by identifying all maximal cliques, finds all points which are in multiple cliques,

Theorem 2. Algorithm (??)

2.3 NP-Completeness Proofs

2.4 Integer Program Foundation

Lemma 7. The graph G defined by Definition (1.3.37) is equivalent to the graph square H^2 defined in Definition (1.3.36)

$$ij \in E_{H^2} \text{ if and only if } 1 \leq d_H(i, j) \leq 2. \quad (2.1)$$

Proof. For $i \neq j$, since D_H is diagonal, have $(D_H)_{ij} = 0$ and hence

$$((A_H)^2 - D_H + A_H)_{ij} = (A_H)_{ij}^2 + (A_H)_{ij}.$$

Suppose $ij \in E_G$ such that $(A_H)_{ij}^2 + (A_H)_{ij} \geq 1$. This implies that at least one of $(A_H)_{ij} = 1$ or $(A_H)_{ij}^2 \geq 1$ must be true. If $(A_H)_{ij} = 1$, then $d_H(i, j) = 1$. If $(A_H)_{ij}^2 \geq 1$, then there exists a vertex k such that $ik, kj \in E_H$, and hence $d_H(i, j) \leq 2$. Therefore $1 \leq d_H(i, j) \leq 2$, and thus $ij \in E_{H^2}$.

Suppose $ij \in E_{H^2}$ such that $1 \leq d_H(i, j) \leq 2$. D_H is diagonal such that $(D_H)_{ij} = 0$. If $d_H(i, j) = 1$, then $(A_H)_{ij} = 1$. If $d_H(i, j) = 2$, then there exists a vertex k such that $ik, kj \in E_H$, and hence $(A_H)_{ij}^2 \geq 1$. In either case, $(A_H)_{ij}^2 + (A_H)_{ij} \geq 1$, so $(A_G)_{ij} = 1$, and hence $ij \in E_G$.

For $i = j$, since all graphs are simple, have $(E)_{ii} = 0$, $(A)_{ii} = 0$ and hence

$$((A_H)^2 - D_H + A_H)_{ii} = (A_H)_{ii}^2 - (D_H)_{ii}.$$

$(A_H)_{ii}^2 - (D_H)_{ii} = 0$ is always true since the degree matrix of H always has diagonal values $(D_H)_{ii} = \sum_k x_{ik}$ and each edge from vertex i to its neighbour in H will always create a loop

1: **Input:** A graph G where $G = T^2$

2: **Output:** A tree T

Step 1

3: **if** $G = K_p$ is complete **then**

4: **Return** tree T with depth 1 where root can be any vertex

5: **else**

6: **Continue**

7: **end if**

Step 2

8: Identify all maximal cliques $C_1, \dots, C_n$ of G using an algorithm such as Bron–Kerbosch where $n > 1$ using characterization (i)

Step 3

9: Let $b_1, \dots, b_n$ be multicliqual points corresponding to $C_1, \dots, C_n$ formed from characterization (iii)

10: Construct a graph S with vertex set $\{b_1, \dots, b_n\}$

11: **for** each pair (C_i, C_j) satisfying $|C_i \cap C_j| = 2$ **do**

12: Add edge between b_i and b_j in S

13: **end for**

14: By characterization (v), S is a tree

Step 4

15: **for** each clique C_i **do**

16: Let n_i be the number of unicliqual points in C_i

17: Attach n_i leaf nodes to b_i in S

18: **end for**

19: The resulting tree T has the same number of nodes as G and satisfies $T^2 = G$

to itself in the square such that $(A_H)_{ii}^2 = \sum_{k=1} x_{ik}x_{ki} = \sum_{k=1} x_{ik}^2$ by the symmetry of adjacency matrix A_H . Since $x_{ik} \in \{0, 1\}$, the square of the boolean is identical to the boolean x_{ik} such that $(A_H)_{ii}^2 = \sum_k x_{ik} = (D_H)_{ii}$ and $(A_G)_{ii} = \mathbb{1}_{\geq 1}((A_H)^2 - D_H + A_H)$ as required.

Combining both gives that $E_G = E_{H^2}$, and so $G = H^2$. $\square$

The formulation of the adjacency matrix of the graph square in Equation (2.18) is very powerful. However, the presence of the indicator function introduces a nonlinearity that prevents the problem from being expressed as a purely quadratic form, thereby limiting the direct applicability of existing results from the quadratic optimization literature.

Lemma 8. *If there are no K_3 triangle subgraphs or any C_4 square subgraphs in the graph root such that H has girth of length 5, then Equation (2.18) can be simplified to become*

$$A_G = ((A_H)^2 - D_H + A_H). \quad (2.2)$$

Proof. Since the graph root has girth 5 there are no cycles of length 3 or 4 in H . Thus for every pair of vertices $i \neq j \in V$ there is either no cycle that connects i, j or there is a cycle of at least length 5 between them. In all cases $(D_H)_{ij} = 0$. If there is no cycle, then there is at most one path between i, j

- i) If the path is of length 1, then $(A_H)_{ij} = 1, (A_H)_{ij}^2 = 0 \implies (A_G)_{ij} = 0 - 0 + 1 = 1$.
- ii) If the path is of length 2, then $(A_H)_{ij} = 0, (A_H)_{ij}^2 = 1 \implies (A_G)_{ij} = 1 - 0 - 0 = 1$.
- iii) If the path is of length ≥ 3 or a path does not exist, then $(A_H)_{ij} = 0, (A_H)_{ij}^2 = 0 \implies (A_G)_{ij} = 0$.

If there is a cycle of length ≥ 5 between i, j such that there are two paths between i, j

- i) If one path is of length 1, then the other is of length ≥ 4 and hence $(A_H)_{ij} = 1, (A_H)_{ij}^2 = 0 \implies (A_G)_{ij} = 0 - 0 + 1 = 1$.
- ii) If one path is of length 2, then the other is of length ≥ 3 and hence $(A_H)_{ij} = 0, (A_H)_{ij}^2 = 1 \implies (A_G)_{ij} = 1 - 0 - 0 = 1$.
- iii) If the paths are both of length ≥ 3 , then $(A_H)_{ij} = 0, (A_H)_{ij}^2 = 0 \implies (A_G)_{ij} = 0$.

If $i = j$, then $(A_H)_{ii} = 0, (D_H)_{ii} = \deg_H(i)$, and $(A_H)_{ii}^2 = \deg_H(i)$. Thus,

$$(A_H)_{ii}^2 - (D_H)_{ii} + (A_H)_{ii} = \deg_H(i) - \deg_H(i) + 0 = 0 = (A_G)_{ii}.$$

$\square$

Let known constants $g_{ij} \in \{0, 1\}$ come from the adjacency matrix of the square A_G and let $M \in \mathbb{Z}_{>0}$ be a sufficiently large constant. The variables in the program are x_{ij}, y_{ikj}, z_{ij} where x_{ij} form the ij -th entry of the adjacency matrix of the root A_H .

2.4.1 Integer Program Formulation

Variables:

$$x_{ij} \in \mathbb{Z}_{\geq 0}, \quad \forall i, j \in \{1, \dots, n\}, \quad (2.3)$$

$$y_{ikj} \in \mathbb{Z}_{\geq 0}, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (2.4)$$

$$z_{ij} \in \mathbb{Z}_{\geq 0}, \quad \forall i, j \in \{1, \dots, n\}. \quad (2.5)$$

Constraints:

$$(1) \text{ Indicator argument: } z_{ii} = \sum_{k=1}^n y_{iki} - \sum_{k=1}^n x_{ik}, \forall i \in \{1, \dots, n\}, \quad (2.6)$$

$$z_{ij} = \sum_{k=1}^n y_{ikj} + x_{ij}, \quad \forall i \neq j \in \{1, \dots, n\}, \quad (2.7)$$

$$(2) \text{ Indicator function: } z_{ij} \leq M g_{ij}, \quad \forall i, j \in \{1, \dots, n\}, \quad (2.8)$$

$$z_{ij} \geq g_{ij}, \quad \forall i, j \in \{1, \dots, n\}, \quad (2.9)$$

$$(3) \text{ Product: } y_{ikj} \leq x_{ik}, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (2.10)$$

$$y_{ikj} \leq x_{kj}, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (2.11)$$

$$y_{ikj} \geq x_{ik} + x_{kj} - 1, \quad \forall i, k, j \in \{1, \dots, n\} \quad (2.12)$$

$$(4) \text{ Upper Bound: } x_{ij} \leq 1, \quad \forall i, j \in \{1, \dots, n\}, \quad (2.13)$$

$$y_{ikj} \leq 1, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (2.14)$$

$$(5) \text{ Symmetry: } x_{ij} = x_{ji}, \quad \forall i, j \in \{1, \dots, n\}, \quad (2.15)$$

$$(6) \text{ Diagonal: } x_{ii} = 0, \quad \forall i \in \{1, \dots, n\}. \quad (2.16)$$

2.4.2 Validity

Define the edge set of H^2 as

$$E_{H^2} = \{uv \mid u \neq v, 1 \leq d_H(u, v) \leq 2\}. \quad (2.17)$$

Then for

$$A_G = \mathbb{1}_{\geq 1}((A_H)^2 - D_H + A_H). \quad (2.18)$$

Lemma 9. *The graph G defined by (2.18) is equivalent to the graph square H^2 defined in (2.17)*

$$ij \in E_{H^2} \text{ if and only if } 1 \leq d_H(i, j) \leq 2. \quad (2.19)$$

Proof. For $i \neq j$, since D_H is diagonal, have $(D_H)_{ij} = 0$ and hence

$$((A_H)^2 - D_H + A_H)_{ij} = (A_H)^2_{ij} + (A_H)_{ij}.$$

Suppose $ij \in E_G$ such that $(A_H)^2_{ij} + (A_H)_{ij} \geq 1$. This implies that at least one of $(A_H)_{ij} = 1$ or $(A_H)^2_{ij} \geq 1$ must be true. If $(A_H)_{ij} = 1$, then $d_H(i, j) = 1$. If $(A_H)^2_{ij} \geq 1$, then there exists a vertex k such that $ik, kj \in E_H$, and hence $d_H(i, j) \leq 2$. Therefore $1 \leq d_H(i, j) \leq 2$, and thus $ij \in E_{H^2}$.

Suppose $ij \in E_{H^2}$ such that $1 \leq d_H(i, j) \leq 2$. D_H is diagonal such that $(D_H)_{ij} = 0$. If $d_H(i, j) = 1$, then $(A_H)_{ij} = 1$. If $d_H(i, j) = 2$, then there exists a vertex k such that $ik, kj \in E_H$, and hence $(A_H)^2_{ij} \geq 1$. In either case, $(A_H)^2_{ij} + (A_H)_{ij} \geq 1$, so $(A_G)_{ij} = 1$, and hence $ij \in E_G$.

For $i = j$, since all graphs are simple, have $(E)_{ii} = 0$, $(A)_{ii} = 0$ and hence

$$((A_H)^2 - D_H + A_H)_{ii} = (A_H)_{ii}^2 - (D_H)_{ii}.$$

$(A_H)_{ii}^2 - (D_H)_{ii} = 0$ is always true since the degree matrix of H always has diagonal values $(D_H)_{ii} = \sum_k x_{ik}$ and each edge from vertex i to its neighbour in H will always create a loop to itself in the square such that $(A_H)_{ii}^2 = \sum_{k=1} x_{ik}x_{ki} = \sum_{k=1} x_{ik}^2$ by the symmetry of adjacency matrix A_H . Since $x_{ik} \in \{0, 1\}$, the square of the boolean is identical to the boolean x_{ik} such that $(A_H)_{ii}^2 = \sum_k x_{ik} = (D_H)_{ii}$ and $(A_G)_{ii} = \mathbb{1}_{\geq 1}((A_H)^2 - D_H + A_H)$ as required.

Combining both gives that $E_G = E_{H^2}$, and so $G = H^2$. $\square$

Theorem 3. *Let G be a graph and consider the integer program whose variables $x_{ij} \in \{0, 1\}$ encode a graph H with adjacency matrix $A_H = (x_{ij})$. Then G has a square root H if and only if the integer program has a feasible solution x such that the corresponding graph H satisfies $H^2 = G$.*

Proof. Suppose G has a square root H and let A_H be the adjacency matrix of H . $x_{ij} = (A_H)_{ij}$. Construct the variables x_{ij}, y_{ikj}, z_{ij} and the constant g_{ij} as such

- i) Let $x_{ij} = 1$ if $ij \in E_H$ and $= 0$ otherwise.
- ii) Let $y_{ikj} = x_{ik}x_{kj}$ for all i, k, j .
- iii) Let $z_{ij} = \sum_{k=1}^n y_{ikj} + x_{ij}$ for all i, j .
- iv) Let $z_{ii} = \sum_{k=1}^n y_{iki} - \sum_{k=1}^n x_{ik} = \sum_{k=1}^n x_{ik}^2 - \sum_{k=1}^n x_{ik} = 0$ since $x_{ij} \in \{0, 1\} \implies x_{ik}^2 = x_{ik}$.
- v) Let $g_{ij} = 1$ if $ij \in E_G$ and $= 0$ otherwise.

Verifying constraints of the integer program for the variables

- i) As $x_{ij} \in \{0, 1\} \implies y_{ikj} \in \{0, 1\}$, Equations (2.3)-(2.5) and Equations (2.13)-(2.14) are satisfied.
- ii) As H is a simple directed graph, its adjacency matrix is hallow and symmetric such that Equations (2.15) -(2.16) are satisfied.
- iii) As $y_{ikj} = x_{ik}x_{kj}$ and all variables are binary, Equations (2.10)- (2.12) are satisfied.
- iv) Equation (2.7) by definition is satisfied.
- v) As $z_{ii} = \sum_{k=1}^n x_{ik}x_{ki} - \sum_{k=1}^n x_{ik}$, Equation (2.6) is satisfied.
- vi) As $z_{ij} \geq 1$ if and only if $x_{ij} = 1$ or there exists k such that $x_{ik} = x_{kj} = 1$ and as $H^2 = G$, have $g_{ij} = 1$ if and only if $1 \leq d_H(i, j) \leq 2$. Thus $g_{ij} = 1$ if and only if $z_{ij} \geq 1$ so the indicator constraints Equations (2.8)-(2.9) hold.

All constraints of the integer program are satisfied by the constructed variables. Hence, the integer program is feasible.

Suppose the integer program on graph G has a feasible solution x , and let x_{ij} define a graph H by

$$ij \in E_H \text{ if and only if } x_{ij} = 1.$$

As x_{ij} satisfies the constraints of the integer program,

- i) Equation (2.3) and Equation (2.13) gives $x_{ij} \in \{0, 1\}$ for all i, j ,
- ii) Equation (2.15) and Equation (2.16) give $x_{ij} = x_{ji}$ and $x_{ii} = 0$,

such that x_{ij} defines a hollow, symmetric matrix with all elements being either 0 or 1 which is a valid adjacency matrix of a graph. Examining the auxiliary variables y_{ikj} and z_{ij} ,

- i) Equations (2.10)-(2.12) give the strict equality $y_{ikj} = x_{ik}x_{kj}$ for all i, j, k ,
- ii) Equation (2.7) gives $z_{ij} = \sum_{k=1}^n y_{ikj} + x_{ij} = \sum_{k=1}^n x_{ik}x_{kj} + x_{ij}$ for all $i \neq j$,
- iii) Equation (2.6) gives $z_{ii} = \sum_{k=1}^n y_{iki} + \sum_{k=1}^n x_{ik} = \sum_{k=1}^n x_{ik}x_{ki} - \sum_{k=1}^n x_{ik} = 0$ for all i since

$$\sum_{k=1}^n x_{ik}x_{ki} = \sum_{k=1}^n x_{ik}^2 = \sum_{k=1}^n x_{ik},$$

by symmetry and since $x_{ij} \in \{0, 1\}$.

The indicator constraints Equations (2.8)-(2.9) thus give for $i \neq j$

$$g_{ij} = 1 \text{ if and only if } \sum_{k=1}^n x_{ik}x_{kj} + x_{ij} \geq 1 \text{ and } g_{ij} = 0 \text{ if and only if } \sum_{k=1}^n x_{ik}x_{kj} + x_{ij} = 0.$$

Since $x_{ij} \in \{0, 1\}$, this is equivalent to

$$g_{ij} = 1 \text{ if and only if } (x_{ij} = 1) \vee (\text{there exists } k \text{ such that } x_{ik} = 1 \wedge x_{kj} = 1) \text{ and } g_{ij} = 0 \text{ if and only if } x_{ij} = 0.$$

By definition of H this means that $g_{ij} = 1$ if and only if $(ij \in E_H) \vee (\text{there exists } k \text{ such that } ik, kj \in E_H)$ which is equivalent to

$$g_{ij} = 1 \text{ if and only if } 1 \leq d_H(i, j) \leq 2.$$

Applying Lemma 9 thus gives $ij \in E_G$ if and only if $ij \in E_H^2$ such that $G = H^2$. □

2.5 Linear Program Relaxation

2.5.1 Formulation

$$x_{ij} \in \mathbb{R}_{\geq 0}, \quad \forall i, j \in \{1, \dots, n\}, \quad (2.20)$$

$$y_{ikj} \in \mathbb{R}_{\geq 0}, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (2.21)$$

$$z_{ij} \in \mathbb{R}_{\geq 0}, \quad \forall i, j \in \{1, \dots, n\}, \quad (2.22)$$

Constraints:

$$(1) \text{ Indicator argument: } z_{ii} = \sum_{k=1}^n y_{iki} - \sum_{k=1}^n x_{ik}, \forall i \in \{1, \dots, n\}, \quad (2.23)$$

$$z_{ij} = \sum_{k=1}^n y_{ikj} + x_{ij}, \quad \forall i \neq j \in \{1, \dots, n\}, \quad (2.24)$$

$$(2) \text{ Indicator function: } z_{ij} \leq M g_{ij}, \quad \forall i, j \in \{1, \dots, n\}, \quad (2.25)$$

$$z_{ij} \geq g_{ij}, \quad \forall i, j \in \{1, \dots, n\}, \quad (2.26)$$

$$(3) \text{ Product: } y_{ikj} \leq x_{ik}, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (2.27)$$

$$y_{ikj} \leq x_{kj}, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (2.28)$$

$$y_{ikj} \geq x_{ik} + x_{kj} - 1, \quad \forall i, k, j \in \{1, \dots, n\} \quad (2.29)$$

$$(4) \text{ Upper Bound: } x_{ij} \leq 1, \quad \forall i, j \in \{1, \dots, n\}, \quad (2.30)$$

$$y_{ikj} \leq 1, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (2.31)$$

$$(5) \text{ Symmetry: } x_{ij} = x_{ji}, \quad \forall i, j \in \{1, \dots, n\}, \quad (2.32)$$

$$(6) \text{ Diagonal: } x_{ii} = 0, \quad \forall i \in \{1, \dots, n\}. \quad (2.33)$$

2.6 Max Linear Program

2.6.1 Formulation

$$x_{ij} \in \mathbb{R}_{\geq 0}, \quad \forall i, j \in \{1, \dots, n\}, \quad (2.34)$$

$$y_{ikj} \in \mathbb{R}_{\geq 0}, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (2.35)$$

$$z_{ij} \in \mathbb{R}_{\geq 0}, \quad \forall i, j \in \{1, \dots, n\}, \quad (2.36)$$

Constraints:

$$(1) \text{ Indicator argument: } z_{ii} = \sum_{k=1}^n y_{iki} - \sum_{k=1}^n x_{ik}, \forall i \in \{1, \dots, n\}, \quad (2.37)$$

$$z_{ij} = \sum_{k=1}^n y_{ikj} + x_{ij}, \quad \forall i \neq j \in \{1, \dots, n\}, \quad (2.38)$$

$$(2) \text{ Indicator function: } z_{ij} \leq M g_{ij}, \quad \forall i, j \in \{1, \dots, n\}, \quad (2.39)$$

$$z_{ij} \geq g_{ij}, \quad \forall i, j \in \{1, \dots, n\}, \quad (2.40)$$

$$(3) \text{ Product: } y_{ikj} \leq x_{ik}, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (2.41)$$

$$y_{ikj} \leq x_{kj}, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (2.42)$$

$$y_{ikj} \geq x_{ik} + x_{kj} - 1, \quad \forall i, k, j \in \{1, \dots, n\} \quad (2.43)$$

$$(4) \text{ Upper Bound: } x_{ij} \leq 1, \quad \forall i, j \in \{1, \dots, n\}, \quad (2.44)$$

$$y_{ikj} \leq 1, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (2.45)$$

$$(5) \text{ Symmetry: } x_{ij} = x_{ji}, \quad \forall i, j \in \{1, \dots, n\}, \quad (2.46)$$

$$(6) \text{ Diagonal: } x_{ii} = 0, \quad \forall i \in \{1, \dots, n\}, \quad (2.47)$$

Objective Function:

$$\max \sum_{i=1}^n \sum_{j=1}^n x_{ij}. \quad (2.48)$$

2.6.2 Fundamental Properties

Proposition 1. Consider the graph square G where the edge between vertices a and b is missing, $ab \notin E_G$ such that $g_{ab} = 0$. Then every feasible solution of $\mathcal{LP}(G)$ satisfies $x_{ab} = 0$ and $y_{avb} = 0$ for all vertices $v \in V \setminus \{a, b\}$.

Proof. Using Equations (2.25)–(2.26) if $g_{ab} = 0$ then $z_{ab} \leq 0$ and $z_{ab} \geq 0$ such that $z_{ab} = 0$. Then Equation (2.24) gives that $0 = \sum_{v=1}^n y_{avb} + x_{ab}$. Since each x, y variable are non-negative, this only holds when each variable is equal to 0 which gives that $x_{ab} = 0$ and $y_{avb} = 0$ for all vertices $v \in V \setminus \{a, b\}$. $\square$

Proposition 2. Consider the graph square G where the edge between vertices a and b is missing, $ab \notin E_G$ such that $g_{ab} = 0$. Then every feasible solution of $\mathcal{LP}(G)$ satisfies that for all vertices $v \in V \setminus \{a, b\}$, $x_{av} + x_{vb} \leq 1$.

Proof. Since Proposition (1) gives $y_{avb} = 0$, using Equation (2.26) gives that $0 = y_{avb} \geq x_{av} + x_{vb} - 1$ which after rearranging gives $x_{av} + x_{vb} \leq 1$ as required. $\square$

Proposition 3. *Consider the graph square G where $ab \in E_G$ but it is known that ab cannot be in the root H , that is $x_{ab} = 0$. Then every feasible solution of $\mathcal{LP}(G)$ satisfies*

Proof.

□

Chapter 3

Square of A Tree using Linear Programming

3.1 Single Triangular Decomposition

3.1.1 Formulation

$$x_{ij} \in \mathbb{R}_{\geq 0}, \quad \forall i, j \in \{1, \dots, n\}, \quad (3.1)$$

$$y_{ikj} \in \mathbb{R}_{\geq 0}, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (3.2)$$

$$z_{ij} \in \mathbb{R}_{\geq 0}, \quad \forall i, j \in \{1, \dots, n\}, \quad (3.3)$$

Constraints:

$$(1) \text{ Indicator argument: } z_{ii} = \sum_{k=1}^n y_{iki} - \sum_{k=1}^n x_{ik}, \forall i \in \{1, \dots, n\}, \quad (3.4)$$

$$z_{ij} = \sum_{k=1}^n y_{ikj} + x_{ij}, \quad \forall i \neq j \in \{1, \dots, n\}, \quad (3.5)$$

$$(2) \text{ Indicator function: } z_{ij} \leq M g_{ij}, \quad \forall i, j \in \{1, \dots, n\}, \quad (3.6)$$

$$z_{ij} \geq g_{ij}, \quad \forall i, j \in \{1, \dots, n\}, \quad (3.7)$$

$$(3) \text{ Product: } y_{ikj} \leq x_{ik}, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (3.8)$$

$$y_{ikj} \leq x_{kj}, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (3.9)$$

$$y_{ikj} \geq x_{ik} + x_{kj} - 1, \quad \forall i, k, j \in \{1, \dots, n\} \quad (3.10)$$

$$(4) \text{ Upper Bound: } x_{ij} \leq 1, \quad \forall i, j \in \{1, \dots, n\}, \quad (3.11)$$

$$y_{ikj} \leq 1, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (3.12)$$

$$(5) \text{ Symmetry: } x_{ij} = x_{ji}, \quad \forall i, j \in \{1, \dots, n\}, \quad (3.13)$$

$$(6) \text{ Diagonal: } x_{ii} = 0, \quad \forall i \in \{1, \dots, n\}, \quad (3.14)$$

Objective Function:

$$\max \sum_{i=1}^n \sum_{j=1}^n x_{ij}. \quad (3.15)$$

3.1.2 Validity

Lemma 10. *Let s, u, v form a triangle in square G where s only lies in a single clique. If $\deg_G(u) = \deg_G(v) = d \geq 2$, then*

$$N_G(u) \setminus \{s\} = N_G(v) \setminus \{s\}.$$

That is, $N_G(u) \cup \{u\}$ and $N_G(v) \cup \{v\}$ induce the same clique structure in G .

Proof. Since s lies in a single clique in G , in any tree root H the vertex s must be a leaf, and exactly one of u or v is the internal vertex adjacent to the remainder of the graph. Assume without loss of generality that u is this internal vertex. Any vertex adjacent to v in G other than s must arise from a path of length at most two through u in H , since v has no other connections in the root besides u and s . Hence

$$N_G(v) \setminus \{s\} \subseteq N_G(u) \setminus \{s\}.$$

Since $\deg_G(u) = \deg_G(v)$, both neighbourhoods have equal cardinality $|N_G(v)| = |N_G(u)|$ such that via the equality criterion for finite sets

$$N_G(v) \setminus \{s\} = N_G(u) \setminus \{s\}.$$

Hence the vertices adjacent to u and v outside of s have identical adjacency structure in G , and therefore $N_G(u) \cup \{u\}$ and $N_G(v) \cup \{v\}$ induce the same clique structure. $\square$

Proposition 4. *Let s, u, v form a triangle in square G where s only lies in a single clique. If $\deg_G(u) = \deg_G(v) = d \geq 2$, then one can choose to assign $x_{su} = x_{uv} = 1$ in the tree root H without loss of generality.*

Proof. Since s lies in a single clique in G , in the root H , s must be a leaf with either u or v being the internal vertex adjacent to the remainder of the graph. Assume without loss of generality that u is the internal vertex.

By Lemma 10, $N_G(u) \setminus \{s\} = N_G(v) \setminus \{s\}$, such that u and v induce identical clique structure in G . Therefore exchanging the roles of u and v preserves the square graph G . Hence choosing u as the internal vertex and assigning

$$x_{su} = x_{uv} = 1,$$

does not alter feasibility, since the alternative choice obtained by exchanging u and v produces an isomorphic tree root with identical square graph. $\square$

Proposition 5. *Let s, u, v form a triangle in square G where s only lies in a single clique. The linear program with Equation (??) never returns a infeasible solution for squares with tree roots.*

Proof. $\square$

3.2 Tetrahedron Decomposition

3.2.1 Formulation

$$x_{ij} \in \mathbb{R}_{\geq 0}, \quad \forall i, j \in \{1, \dots, n\}, \quad (3.16)$$

$$y_{ikj} \in \mathbb{R}_{\geq 0}, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (3.17)$$

$$z_{ij} \in \mathbb{R}_{\geq 0}, \quad \forall i, j \in \{1, \dots, n\}, \quad (3.18)$$

Constraints:

$$(1) \text{ Indicator argument: } z_{ii} = \sum_{k=1}^n y_{iki} - \sum_{k=1}^n x_{ik}, \forall i \in \{1, \dots, n\}, \quad (3.19)$$

$$z_{ij} = \sum_{k=1}^n y_{ikj} + x_{ij}, \quad \forall i \neq j \in \{1, \dots, n\}, \quad (3.20)$$

$$(2) \text{ Indicator function: } z_{ij} \leq M g_{ij}, \quad \forall i, j \in \{1, \dots, n\}, \quad (3.21)$$

$$z_{ij} \geq g_{ij}, \quad \forall i, j \in \{1, \dots, n\}, \quad (3.22)$$

$$(3) \text{ Product: } y_{ikj} \leq x_{ik}, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (3.23)$$

$$y_{ikj} \leq x_{kj}, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (3.24)$$

$$y_{ikj} \geq x_{ik} + x_{kj} - 1, \quad \forall i, k, j \in \{1, \dots, n\} \quad (3.25)$$

$$(4) \text{ Upper Bound: } x_{ij} \leq 1, \quad \forall i, j \in \{1, \dots, n\}, \quad (3.26)$$

$$y_{ikj} \leq 1, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (3.27)$$

$$(5) \text{ Symmetry: } x_{ij} = x_{ji}, \quad \forall i, j \in \{1, \dots, n\}, \quad (3.28)$$

$$(6) \text{ Diagonal: } x_{ii} = 0, \quad \forall i \in \{1, \dots, n\}, \quad (3.29)$$

Objective Function:

$$\max \sum_{i=1}^n \sum_{j=1}^n x_{ij}. \quad (3.30)$$

3.2.2 Validity

Lemma 11. *Let s_1, s_2, u, v form a 4-clique in square G . If $N_G(s_1) \setminus \{s_1, s_2, u, v\} = N_G(s_2) \setminus \{s_1, s_2, u, v\}$, then*

$$N_G(u) \setminus \{s_1, s_2\} = N_G(v) \setminus \{s_1, s_2\}.$$

That is, $N_G(u) \cup \{u\}$ and $N_G(v) \cup \{v\}$ induce the same clique structure in G .

Proof. Let $x \in N_G(v) \setminus \{s_1, s_2\}$ be a vertex adjacent to v in G such that there exists a path of length at most two between x and v in H . If x is adjacent to either s_1 or s_2 , then by the assumption

$$N_G(s_1) \setminus \{s_1, s_2, u, v\} = N_G(s_2) \setminus \{s_1, s_2, u, v\},$$

where the vertices s_1 and s_2 contribute identically outside the clique. Hence any external connection from the clique through v must also be realised through the same local structure containing u . Consequently x is also at distance at most two from u in H , and therefore

$$x \in N_G(u) \setminus \{s_1, s_2\},$$

which implies that $N_G(v) \setminus \{s_1, s_2\} \subseteq N_G(u) \setminus \{s_1, s_2\}$. By symmetry, interchanging the roles of u and v gives

$$N_G(u) \setminus \{s_1, s_2\} \subseteq N_G(v) \setminus \{s_1, s_2\}$$

and hence $N_G(u) \setminus \{s_1, s_2\} = N_G(v) \setminus \{s_1, s_2\}$. Therefore the vertices adjacent to u and v outside of $\{s_1, s_2\}$ have identical adjacency structure in G , and so $N_G(u) \cup \{u\}$ and $N_G(v) \cup \{v\}$ induce the same clique structure in G . $\square$

Proposition 6. *Let s_1, s_2, u, v form a 4-clique in square G . If $\deg_G(u) = \deg_G(v) = d \geq 3$, then one can choose to assign $x_{s_1u} = x_{s_2u} = x_{uv} = 1$ in the tree root H without loss of generality.*

Proof. $\square$

Proposition 7. *Let s_1, s_2, u, v form a 4-clique in square G . The linear program with Equation (??) never returns an infeasible solution for squares with tree roots.*

Proof. $\square$

Proposition 8. *Let H be a tree and let u be an internal vertex with leaf children $s_1, \dots, s_k$ and parent v . Fixing any two leaves together with their internal vertex and parent uniquely determines the remaining leaf children of u in the linear program.*

Proof. Since any tree may be rooted at an arbitrary vertex, assume without loss of generality that H is rooted. Let $s_1, \dots, s_k$ be leaf children adjacent to an internal vertex u , and let v be the parent of u . Consider any pair of leaves s_i, s_j , and suppose the internal branching vertex u together with its parent v have been fixed in the linear program. Now consider any remaining leaf s_ℓ adjacent to u . Since s_ℓ is a leaf in the root, its only neighbour in the tree is u . Furthermore, all leaf children of u have identical neighbourhood structure in the square graph outside of $\{u, v, s_i, s_j\}$.

Now consider any remaining leaf s_ℓ adjacent to u . Since every leaf adjacent to u lies at distance two from s_ℓ through u , and v lies at distance two through the path $s_\ell - u - v$, the neighbourhood of s_ℓ in the square graph is

$$N_G(s_\ell) = \{u, v, s_1, \dots, s_{\ell-1}, s_{\ell+1}, \dots, s_k\}.$$

Suppose for contradiction that s_ℓ is assigned to a vertex w with $w \neq u$. Since s_ℓ is a leaf in the rooted tree, its neighbourhood in the square graph is determined by the vertices at distance at most two from w . Since $w \neq u$, the set of vertices at distance at most two from s_ℓ differs from that induced through u , giving

$$N_G(s_\ell) \neq \{u, v, s_1, \dots, s_{\ell-1}, s_{\ell+1}, \dots, s_k\}.$$

Hence either additional adjacencies are introduced into the square graph or existing adjacencies are removed and the neighbourhood of s_ℓ in the resulting square graph differs from that of the original graph, contradicting feasibility of the linear program. Thus the only feasible assignment is for s_ℓ to be adjacent to the already identified internal vertex u , implying that fixing any two leaves together with their internal vertex and parent uniquely determines all remaining leaf children adjacent to u . $\square$

3.3 Tree Proof

3.3.1 Integer Program

Lemma 12. *Consider the graph square G containing vertices i, j and k where edges ik and kj are present and ij is absent in G such that there exists a P_2 . Then every feasible solution of $\mathcal{IP}(G)$ must satisfy*

$$x_{ik} + x_{kj} \leq 1.$$

Proof. As ij is absent in G , $g_{ij} = 0$ such that Equation (2.8) gives $z_{ij} \leq 0$ and Equation (2.9) gives $z_{ij} \geq 0$ such that $z_{ij} = 0$. Therefore Equation (2.7) gives

$$0 = \sum_{a=1}^n y_{iaj} + x_{ij} = \sum_{a=1}^n x_{ia}x_{aj} + x_{ij},$$

where the second equality follows from the product constraints Equations (2.10)–(2.12) together with the integrality of $x \in \{0, 1\}$ by Equation (2.3) and Equation (2.13). Since all variables are non-negative, 0 only holds when $x_{ij} = 0$ and all of $x_{ia}x_{aj} = 0$ and using the integrality of x , this means that at most one of x_{ia}, x_{aj} is equal to 1 and the other must be 0, which can be equivalently written as $x_{ia} + x_{aj} \leq 1$. Taking $a = k$ gives $x_{ik} + x_{kj} \leq 1$ as required. $\square$

3.3.2 Linear Program

Lemma 13. *Consider the graph square G containing vertices i, j and k where edges ik and kj are present and ij is absent in G such that there exists a P_2 . Then every feasible solution of $\mathcal{LP}(G)$ must satisfy*

$$x_{ik} + x_{kj} \leq 1.$$

Proof. Since $g_{ij} = 0$, Equations (2.25)–(2.26) imply that $z_{ij} = 0$. Equation (2.24) therefore gives

$$0 = z_{ij} = \sum_{a=1}^n y_{iaj} + x_{ij}.$$

Since all variables are nonnegative by Equations (2.20)–(2.22), every term in the above sum must be equal to zero with $x_{ij} = 0$ and $y_{iaj} = 0$ for all a . Thus applying Equation (2.29) to $y_{ikj} = 0$ gives

$$0 = y_{ikj} \geq x_{ik} + x_{kj} - 1,$$

and hence $x_{ik} + x_{kj} \leq 1$ as required. $\square$

3.3.3 Max Linear Program

Conjecture 1. *Let x be an optimal solution to $\mathcal{MLP}(G)$. If edge $ij \notin E_G$, that is $g_{ij} = 0$, then for every $k \in \{1, \dots, n\}$,*

$$x_{ik} + x_{kj} \in \{0, 1\}.$$

Proof. $\square$

Lemma 14. *Every tree contains either a pendant or a leaf clique subgraph.*

Proof. Let T be a finite tree with $|V(T)| \geq 3$, and root T at an arbitrary vertex. Since T is finite, there exists a leaf s of maximal depth where its parent is u . Every child of u is a leaf, since if there was some child w of u that is not a leaf, then w would have a child and hence there would exist a vertex at depth strictly greater than that of s , contradicting the choice of s .

If u is not the root, let v denote its parent such that every neighbour of u other than v is a leaf.

If u has exactly one leaf neighbour s besides v (when u is not the root), then u has exactly two neighbours, namely s and v , hence $\deg_T(u) = 2$. Therefore the induced subgraph on

$$\{s, u, v\}$$

forms a pendant subgraph.

Otherwise u has at least two leaf neighbours s_1, s_2 . Choose another neighbour v of u distinct from s_1, s_2 ; if u is not the root, take its parent, while if u is the root choose any remaining neighbour. Then

$$\{s_1, s_2, v\} \subseteq N(u),$$

and $\deg_T(u) \geq 3$. Hence the induced subgraph on

$$\{s_1, s_2, u, v\}$$

forms a leaf clique subgraph. □

Proof. Consider the smallest tree of size 3 that is unique up to isomorphism, $s - u - v$ which is clearly a pendant. In order to form a tree without a pendant or a leaf-clique, a vertex a must be added. However, without loss of generality adding a to the leaf v forms a new pendant $\{a, v, u\}$ whilst adding a to the internal vertex u forms a $\{a, u, v\}$ □

Chapter 4

Further Work

In this chapter we discuss additional work, further concepts and interesting ideas around specific structural properties that square intrinsically holds. Most of these have not been rigorously proven, with only sketch proofs given or intuitions given behind why they help improve results.

Further work would naturally be to improve many of these nascent ideas and develop formal proofs.

4.1 Symbolic Groups

Lemma 12 and its linear relaxation equivalent Lemma 13 give extremely strong structure into how the linear program operates. Using the maximization objective function Lemma 13, any two edges ij, jk which exist in G but ki , can be assigned the weights a, b respectively where $x_{ij} = a, x_{jk} = b$ and $a + b \leq 1$.

Under certain conditions, the relationship can hold with equality $a + b = 1$ which therefore gives a methodology for tackling scenarios where the maximal fractional objective function is greater than the optimal integer objective function corresponding to a valid square root. By continuously iterating Lemma 13 can eventually categorize and formulate equations for many of the weights on the various edges.

Consider this graph square on 6 vertices with three different roots

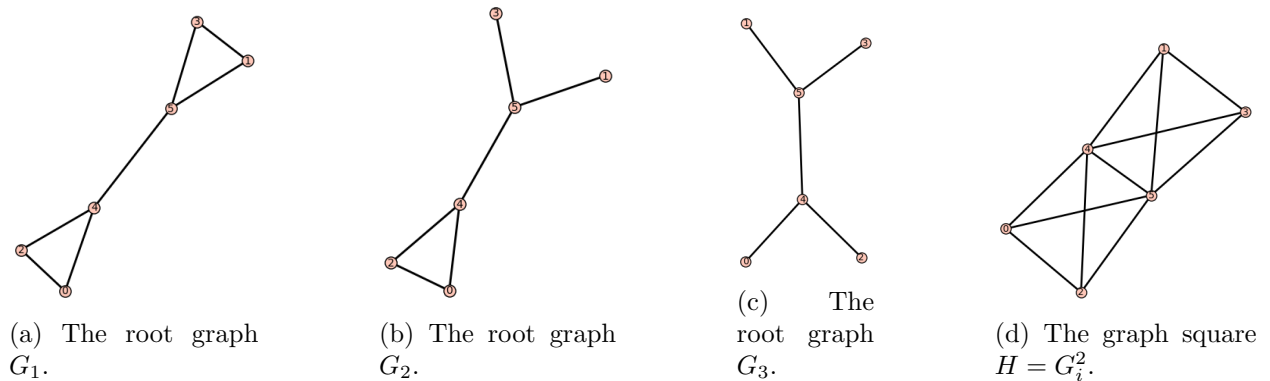


Figure 4.1: Three non-isomorphic square roots of the same six-vertex graph square.

G_1 is the unique maximal root. Assigning $a = x_{14}$ gives using Lemma (??) that $1 - a = x_{40}$ since 01 does not exist in G . The same applies such that $1 - a = x_{42}$ and $a = x_{43}$, whereby the

existence of edges 13 and 02 prevent additional constraints being imposed upon a , it is internally consistent. By symmetry can likewise assign $b = x_{15} = x_{35}$ and $1 - b = x_{50} = x_{52}$. Setting $a = 0$ and $b = 1$ thus gives x_{40}

Consider this graph square on 6 vertices with only one unique root

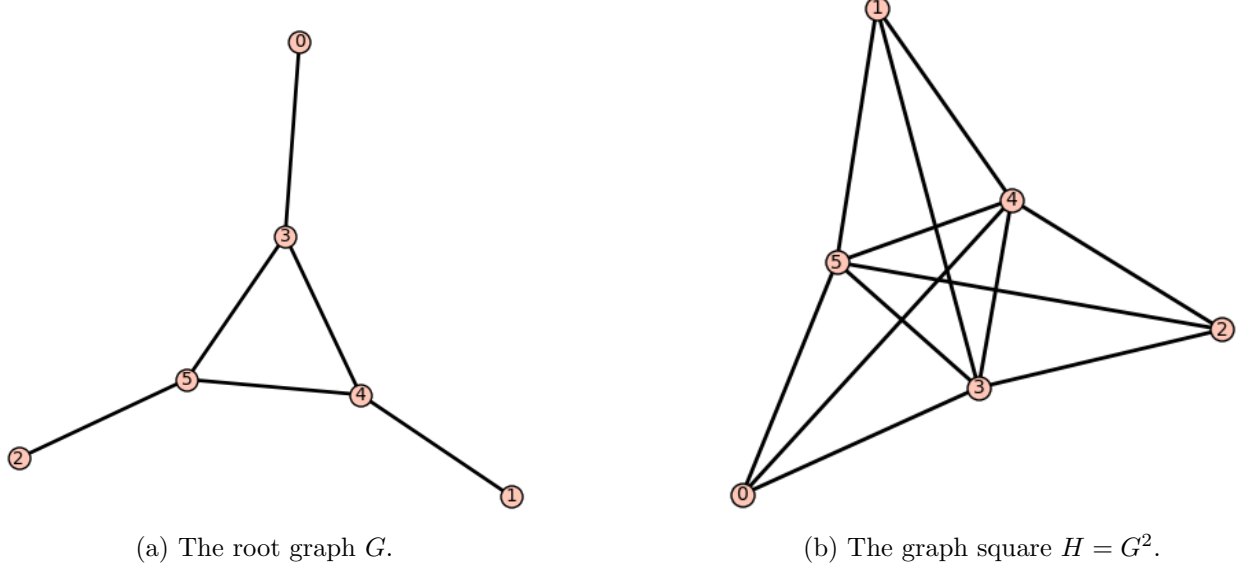


Figure 4.2: A six-vertex graph and its square.

Assigning $a = x_{03}$ gives using Lemma (??) that $x_{31} = 1 - a$ since 01 does not exist in G , and likewise with 12 does not exist in G have $x_{32} = a$. However with the absence of 02, must have $x_{03} + x_{32} \leq 1$ such that $a \leq \frac{1}{2}$. By symmetry this likewise holds for $b = x_{05} = x_{15}$, $1 - b = x_{52}$ and $c = x_{42} = x_{41}$, $1 - c = x_{40}$ giving $b, c \leq \frac{1}{2}$.

In this case, rather than maximizing the objective function and giving a fractional solution, setting $a = b = c = 0$ gives a valid integral solution.

4.2 Duality

Consider the dual formulation of the max objective linear program defined in Formulation 2.6.1 by first restating it in canonical form.

Objective Function:

$$\min - \sum_{i=1}^n \sum_{j=1}^n x_{ij}. \quad (4.1)$$

Variables:

$$x_{ij} \in \mathbb{R}_{\geq 0}, \quad \forall i, j \in \{1, \dots, n\}, \quad (4.2)$$

$$y_{ikj} \in \mathbb{R}_{\geq 0}, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (4.3)$$

$$z_{ij} \in \mathbb{R}_{\geq 0}, \quad \forall i, j \in \{1, \dots, n\}. \quad (4.4)$$

Constraints:

$$(1) \text{ Indicator argument: } z_{ii} - \sum_{k=1}^n y_{iki} + \sum_{k=1}^n x_{ik} - x_{ii} = 0, \forall i \in \{1, \dots, n\}, \quad (4.5)$$

$$z_{ij} - \sum_{k=1}^n y_{ikj} - x_{ij} = 0, \quad \forall i \neq j \in \{1, \dots, n\}, \quad (4.6)$$

$$(2) \text{ Indicator function: } z_{ij} - M g_{ij} \leq 0, \quad \forall i, j \in \{1, \dots, n\}, \quad (4.7)$$

$$-z_{ij} + g_{ij} \leq 0, \quad \forall i, j \in \{1, \dots, n\}, \quad (4.8)$$

$$(3) \text{ Product: } y_{ikj} - x_{ik} \leq 0, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (4.9)$$

$$y_{ikj} - x_{kj} \leq 0, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (4.10)$$

$$x_{ik} + x_{kj} - y_{ikj} - 1 \leq 0, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (4.11)$$

$$(4) \text{ Symmetry: } x_{ij} - x_{ji} = 0, \quad \forall i, j \in \{1, \dots, n\}, \quad (4.12)$$

$$(5) \text{ Diagonal: } x_{ii} = 0, \quad \forall i \in \{1, \dots, n\}, \quad (4.13)$$

$$(6) \text{ Variable Bound: } x_{ij} - 1 \leq 0, \quad \forall i, j \in \{1, \dots, n\}, \quad (4.14)$$

$$y_{ikj} - 1 \leq 0, \quad \forall i, k, j \in \{1, \dots, n\}. \quad (4.15)$$

Its dual can be rewritten as

Objective Function:

$$\max \sum_{i=1}^n \sum_{j=1}^n \left(-M g_{ij} \gamma_{ij} + g_{ij} \delta_{ij} - \lambda_{ij} \right) - \sum_{i=1}^n \sum_{k=1}^n \sum_{j=1}^n (\eta_{ikj} + \mu_{ikj}). \quad (4.16)$$

Dual Variables:

$$\alpha_i \in \mathbb{R}, \quad \forall i \in \{1, \dots, n\}, \quad (\text{associated with (4.5)}), \quad (4.17)$$

$$\beta_{ij} \in \mathbb{R}, \quad \forall i \neq j \in \{1, \dots, n\}, (\text{associated with (4.6)}), \quad (4.18)$$

$$\gamma_{ij} \geq 0, \quad \forall i, j \in \{1, \dots, n\}, \quad (\text{associated with (4.7)}), \quad (4.19)$$

$$\delta_{ij} \geq 0, \quad \forall i, j \in \{1, \dots, n\}, \quad (\text{associated with (4.8)}), \quad (4.20)$$

$$\epsilon_{ikj} \geq 0, \quad \forall i, k, j \in \{1, \dots, n\}, (\text{associated with (4.9)}), \quad (4.21)$$

$$\zeta_{ikj} \geq 0, \quad \forall i, k, j \in \{1, \dots, n\}, (\text{associated with (4.10)}), \quad (4.22)$$

$$\eta_{ikj} \geq 0, \quad \forall i, k, j \in \{1, \dots, n\}, (\text{associated with (4.11)}), \quad (4.23)$$

$$\theta_{ij} \in \mathbb{R}, \quad \forall i, j \in \{1, \dots, n\}, \quad (\text{associated with (4.12)}), \quad (4.24)$$

$$\iota_i \in \mathbb{R}, \quad \forall i \in \{1, \dots, n\}, \quad (\text{associated with (4.13)}), \quad (4.25)$$

$$\lambda_{ij} \geq 0, \quad \forall i, j \in \{1, \dots, n\}, \quad (\text{associated with (4.14)}), \quad (4.26)$$

$$\mu_{ikj} \geq 0, \quad \forall i, k, j \in \{1, \dots, n\}, (\text{associated with (4.15)}). \quad (4.27)$$

Dual Constraints:

$\forall i, j \in \{1, \dots, n\}$, each primal x_{ij} :

$$\begin{cases} -1 - \sum_{k=1}^n \epsilon_{iik} - \sum_{k=1}^n \zeta_{kii} + \sum_{k=1}^n \eta_{iik} + \sum_{k=1}^n \eta_{kii} + \iota_i + \lambda_{ii} \geq 0, & \text{if } j = i, \\ -1 + \alpha_i - \beta_{ij} - \sum_{k=1}^n \epsilon_{ikj} - \sum_{k=1}^n \zeta_{kij} + \sum_{k=1}^n \eta_{ikj} + \sum_{k=1}^n \eta_{kij} + \theta_{ij} - \theta_{ji} + \lambda_{ij} \geq 0, & \text{if } j \neq i, \end{cases} \quad (4.28)$$

$\forall i, k, j \in \{1, \dots, n\}$, each primal y_{ikj} :

$$\begin{cases} -\alpha_i + \epsilon_{iki} + \zeta_{iki} - \eta_{iki} + \mu_{iki} \geq 0, & \text{if } j = i, \\ -\beta_{ij} + \epsilon_{ikj} + \zeta_{ikj} - \eta_{ikj} + \mu_{ikj} \geq 0, & \text{if } j \neq i, \end{cases} \quad (4.29)$$

$\forall i, j \in \{1, \dots, n\}$, each primal z_{ij} :

$$\begin{cases} \alpha_i + \gamma_{ii} - \delta_{ii} = 0, & \text{if } j = i, \\ \beta_{ij} + \gamma_{ij} - \delta_{ij} = 0, & \text{if } j \neq i. \end{cases} \quad (4.30)$$

whereby the Lagrangian is

$$\begin{aligned} \mathcal{L}(x, y, z) = & - \sum_{i=1}^n \sum_{j=1}^n x_{ij} + \sum_{i=1}^n \alpha_i \left(z_{ii} - \sum_{k=1}^n y_{iki} + \sum_{k=1}^n x_{ik} - x_{ii} \right) + \sum_{i \neq j} \beta_{ij} \left(z_{ij} - \sum_{k=1}^n y_{ikj} - x_{ij} \right) \\ & + \sum_{i=1}^n \sum_{j=1}^n \gamma_{ij} (z_{ij} - M g_{ij}) + \sum_{i=1}^n \sum_{j=1}^n \delta_{ij} (-z_{ij} + g_{ij}) + \sum_{i=1}^n \sum_{k=1}^n \sum_{j=1}^n \epsilon_{ikj} (y_{ikj} - x_{ik}) \\ & + \sum_{i=1}^n \sum_{k=1}^n \sum_{j=1}^n \zeta_{ikj} (y_{ikj} - x_{kj}) + \sum_{i=1}^n \sum_{k=1}^n \sum_{j=1}^n \eta_{ikj} (x_{ik} + x_{kj} - y_{ikj} - 1) \\ & + \sum_{i=1}^n \sum_{j=1}^n \theta_{ij} (x_{ij} - x_{ji}) + \sum_{i=1}^n \iota_i x_{ii} + \sum_{i=1}^n \sum_{j=1}^n \lambda_{ij} (x_{ij} - 1) + \sum_{i=1}^n \sum_{k=1}^n \sum_{j=1}^n \mu_{ikj} (y_{ikj} - 1). \end{aligned}$$

Collecting terms gives

$$\begin{aligned} \mathcal{L}(x, y, z) = & \sum_{i=1}^n \sum_{j=1}^n x_{ij} \left[-1 + \mathbf{1}_{\{j=i\}} \left(\alpha_i - \sum_{k=1}^n \epsilon_{iki} - \sum_{k=1}^n \zeta_{kii} + \sum_{k=1}^n \eta_{iik} + \sum_{k=1}^n \eta_{kii} + \iota_i + \lambda_{ii} \right) \right. \\ & \left. + \mathbf{1}_{\{j \neq i\}} \left(\alpha_i - \beta_{ij} - \sum_{k=1}^n \epsilon_{ikj} - \sum_{k=1}^n \zeta_{kij} + \sum_{k=1}^n \eta_{ikj} + \sum_{k=1}^n \eta_{kij} + \lambda_{ij} \right) + \theta_{ij} - \theta_{ji} \right] \\ & + \sum_{i=1}^n \sum_{k=1}^n \sum_{j=1}^n y_{ikj} \left[-\mathbf{1}_{\{j=i\}} \alpha_i - \mathbf{1}_{\{j \neq i\}} \beta_{ij} + \epsilon_{ikj} + \zeta_{ikj} - \eta_{ikj} + \mu_{ikj} \right] \\ & + \sum_{i=1}^n \sum_{j=1}^n z_{ij} \left[\mathbf{1}_{\{j=i\}} \alpha_i + \mathbf{1}_{\{j \neq i\}} \beta_{ij} + \gamma_{ij} - \delta_{ij} \right] \\ & - \sum_{i=1}^n \sum_{j=1}^n M g_{ij} \gamma_{ij} + \sum_{i=1}^n \sum_{j=1}^n g_{ij} \delta_{ij} - \sum_{i=1}^n \sum_{k=1}^n \sum_{j=1}^n \eta_{ikj} - \sum_{i=1}^n \sum_{j=1}^n \lambda_{ij} - \sum_{i=1}^n \sum_{k=1}^n \sum_{j=1}^n \mu_{ikj}. \end{aligned}$$

4.2.1 Rounding

Rounding fractional solutions such that an integral solution or the problem is deemed infeasible gives the classification in Figure 8 is obtained.

Result Diagrams

Linear Program

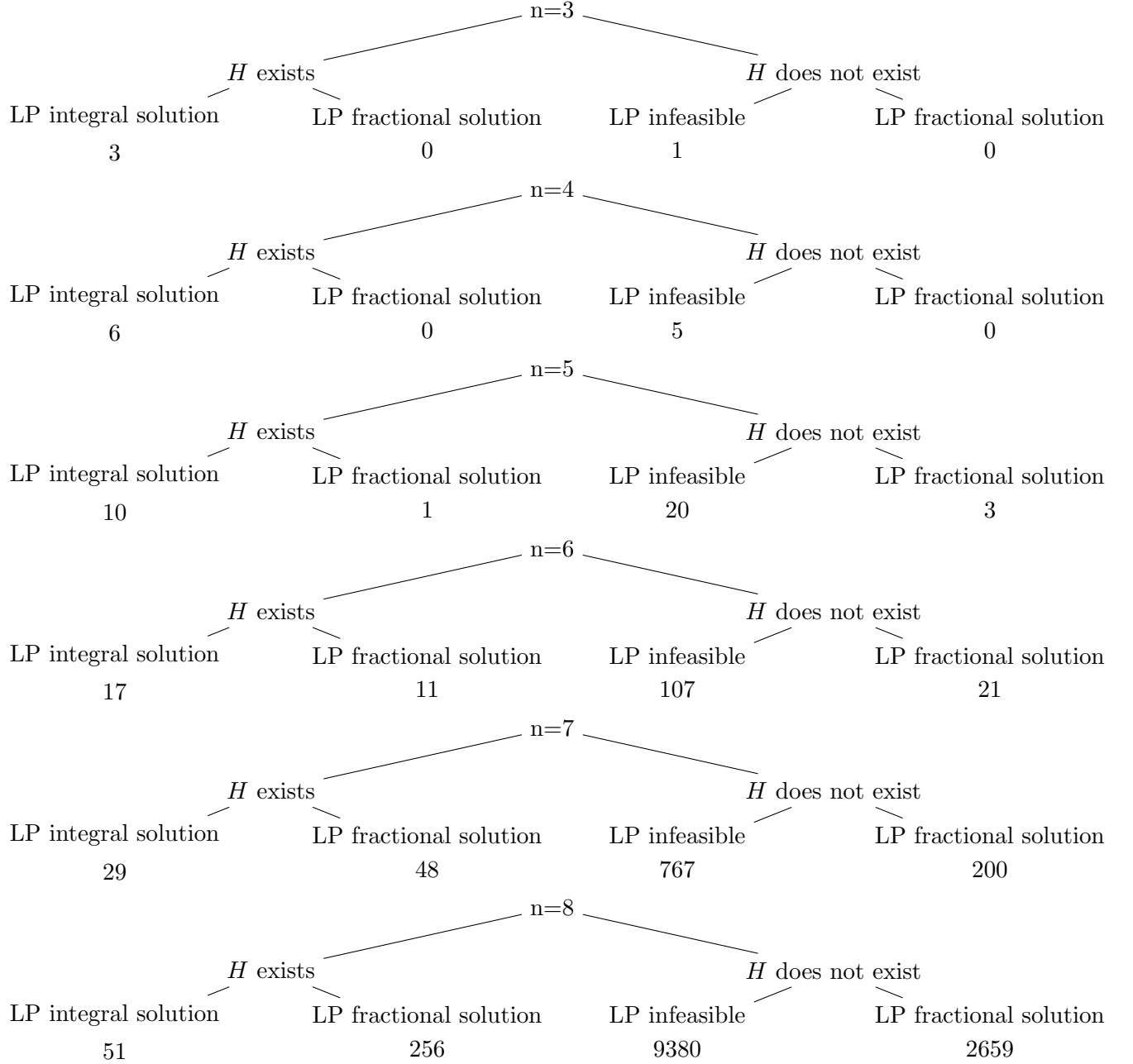


Figure 3: LP outcomes for different values of n on base linear program.

1

¹The total number of simple graphs on $n = 2, 3, 4, \dots$ unlabeled nodes is the sequence 2, 4, 11, 34, 156, 1044, 12346, $\dots$ given by A000088.

The IP integral solution sequence 2, 3, 6, 10, 17, 29, 51, $\dots$ is given by A285665.

The total number of graphs which are squares is the sequence 2, 3, 6, 11, 28, 77, 307, $\dots$ given by A382181.

Max Objective Function

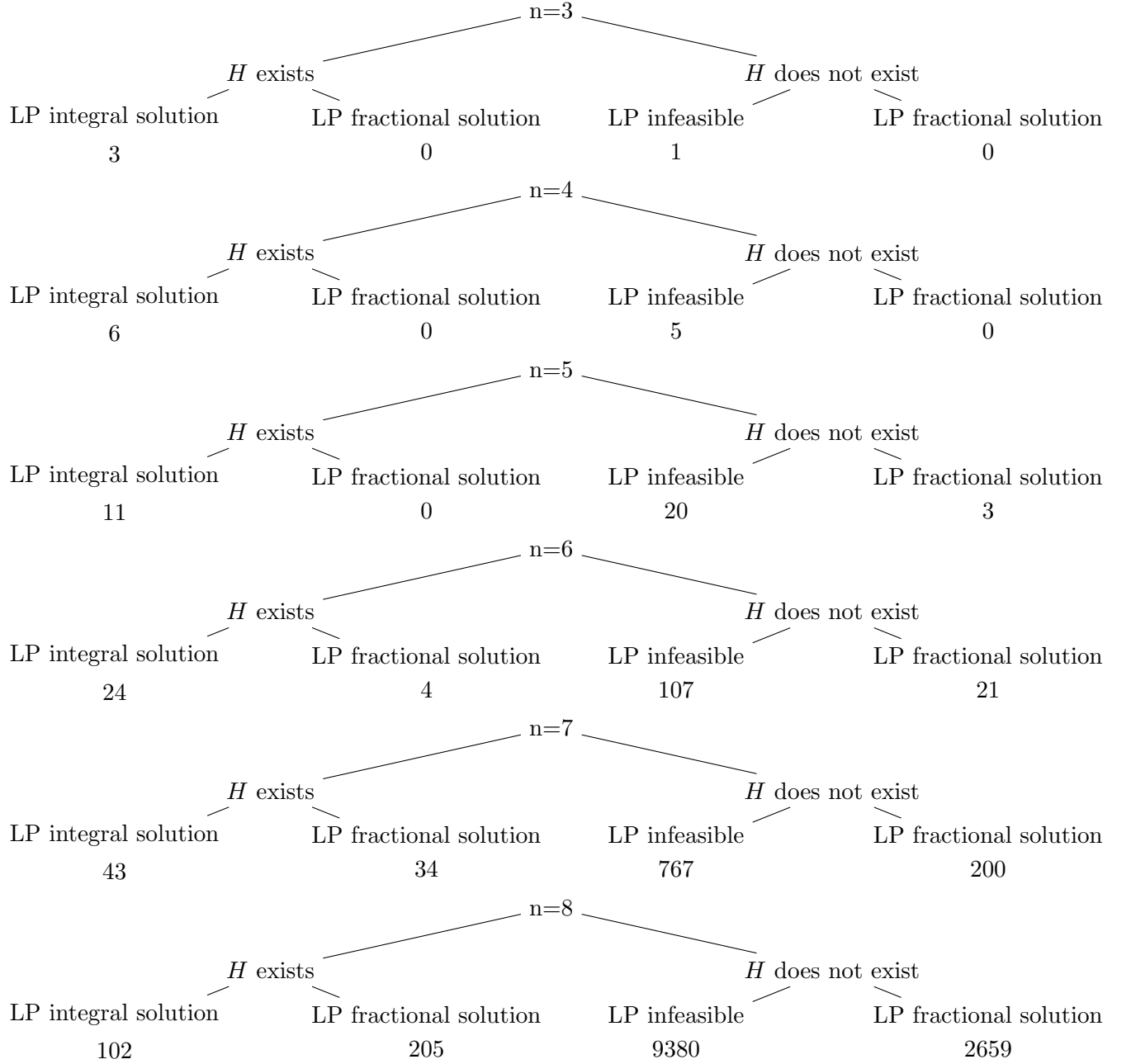


Figure 4: LP outcomes for different values of n on the linear program with max objective function.

2

²The LP integer solution sequence 3, 6, 11, 22, 42, 102, ... is very similar to the number of distinct characteristic polynomials of trees with n nodes - A226594.

Min Objective Function

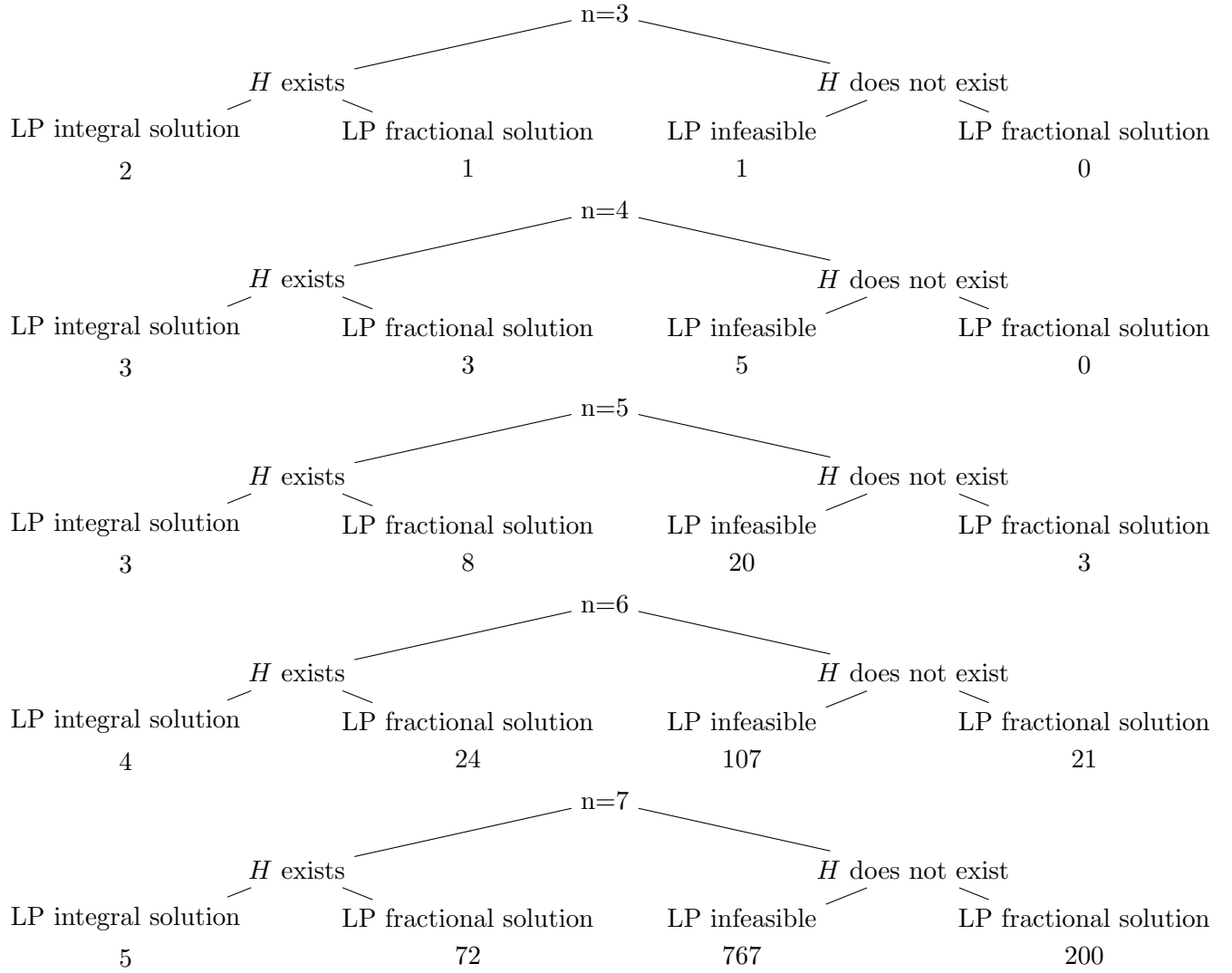


Figure 5: LP outcomes for different values of n on the linear program with min objective function.

Single Triangular Decomposition

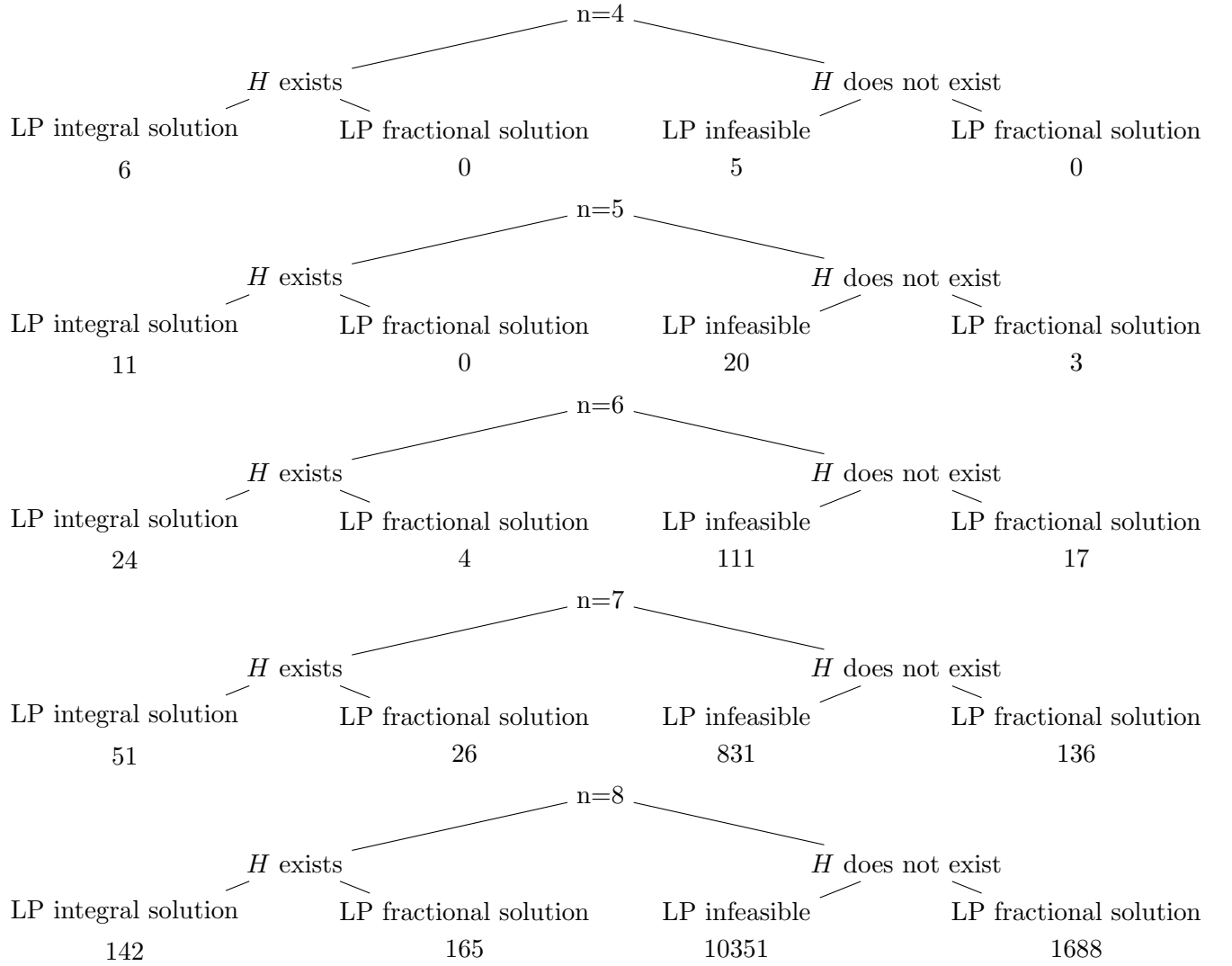


Figure 6: LP outcomes for different values of n on the linear program with max objective function and single triangular decomposition.

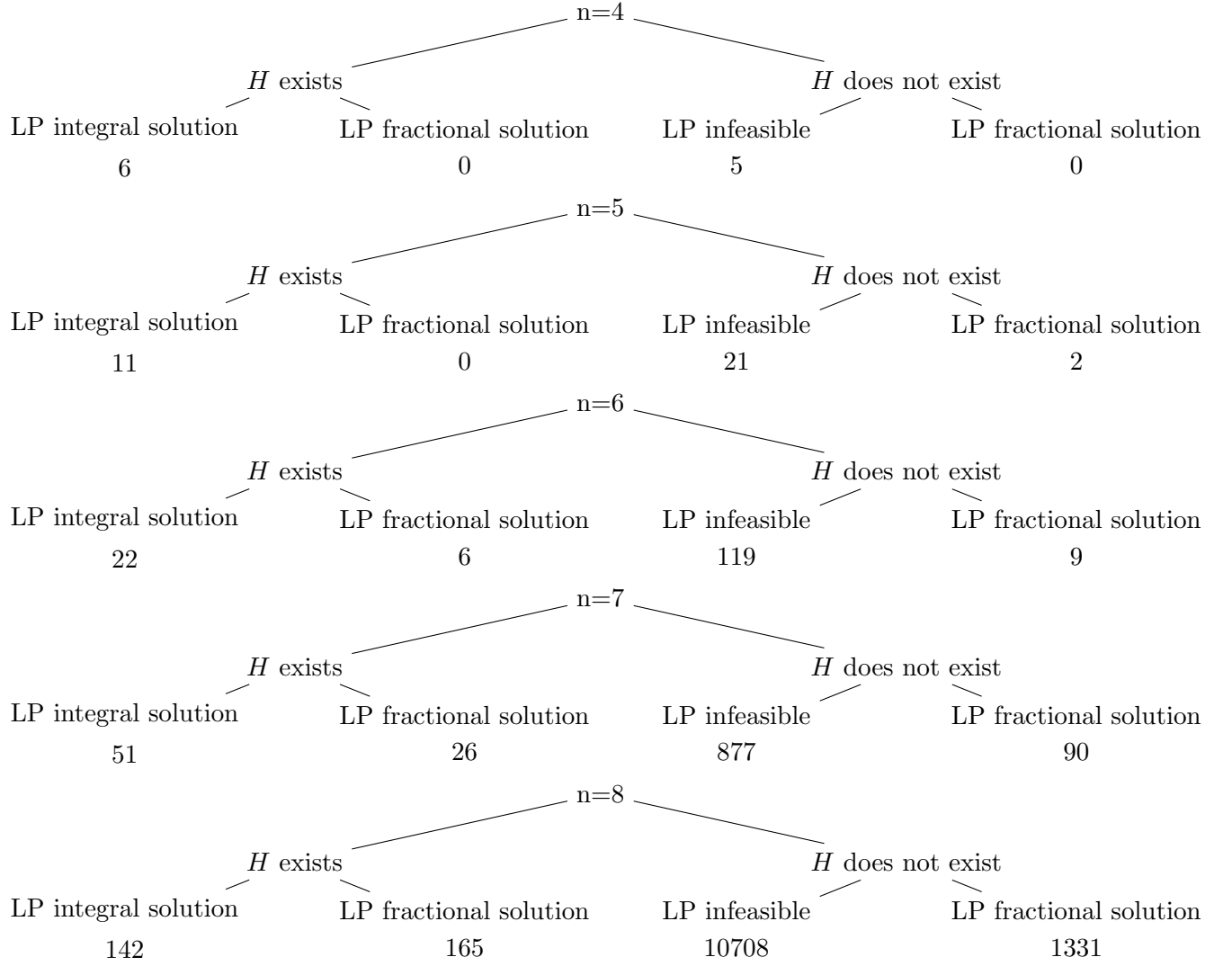


Figure 7: LP outcomes for different values of n on the linear program with max objective function and single triangular decomposition with recursion for tie-breaking.

Rounding

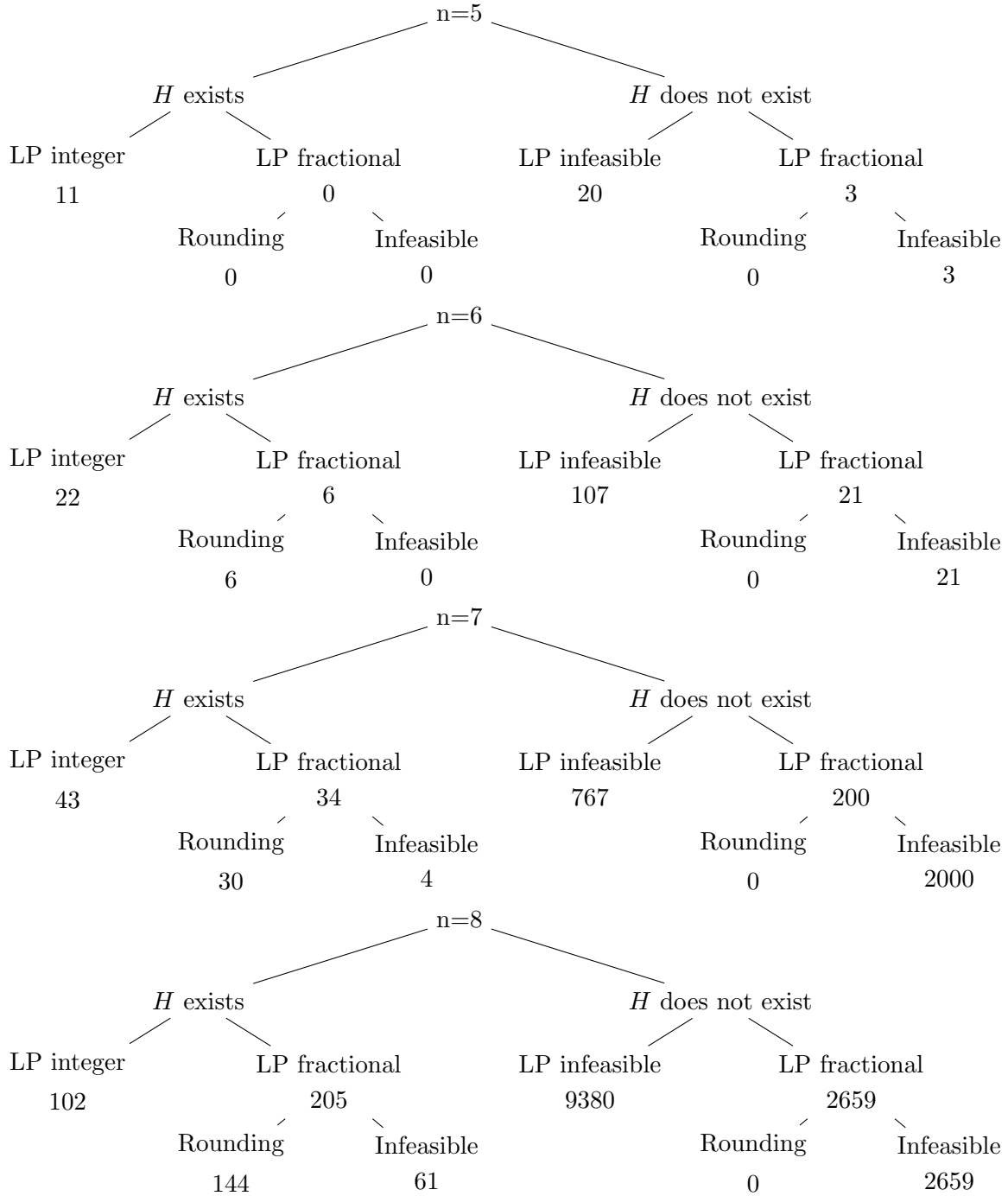


Figure 8: LP outcomes for different values of n on the linear program with max objective function and using rounding for fractional solutions.

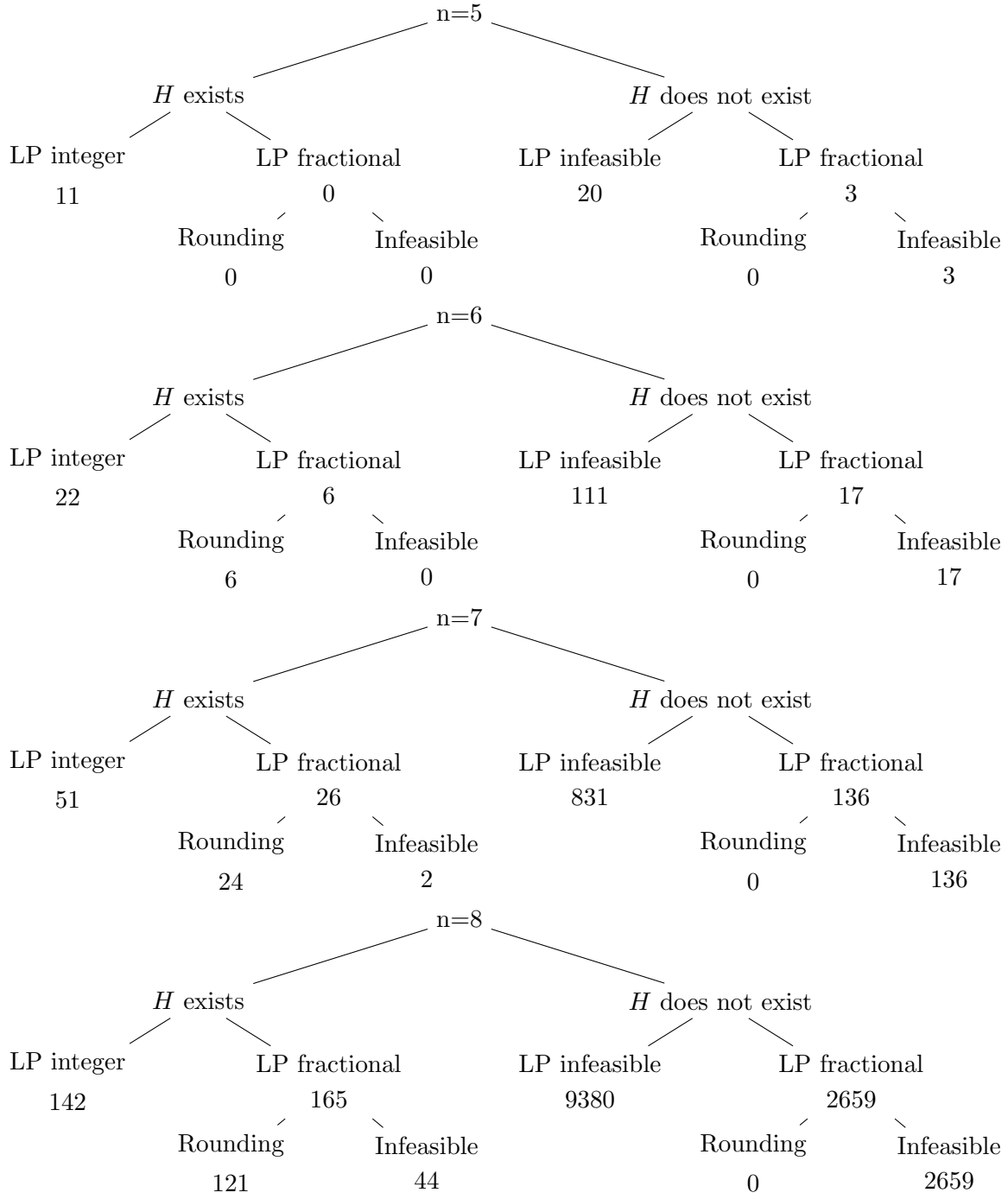


Figure 9: LP outcomes for different values of n on the linear program with max objective function, single triangular decomposition and using rounding for fractional solutions.

Bibliography

- [Coc+13] Manfred Cochefert, Jean-François Couturier, Petr A. Golovach, Dieter Kratsch, and Daniël Paulusma. “Sparse Square Roots”. In: *Lecture Notes in Computer Science* 8165 (2013), pp. 177–188.
- [Duf25] Christopher Duffy. “Quasi-Transitive Mixed Graphs and Undirected Squares of Oriented Graphs”. In: *Australasian Journal of Combinatorics* 92.2 (2025), pp. 142–163.
- [Far+09] Babak Farzad, Lap Chi Lau, Van Bang Le, and Nguyen Ngoc Tuy. “Computing Graph Roots Without Short Cycles”. In: *Proceedings of the 26th International Symposium on Theoretical Aspects of Computer Science (STACS 2009)*. Vol. 3. Leibniz International Proceedings in Informatics (LIPIcs). 2009, pp. 397–408.
- [FBB23] Leon Festinger, Françoise Bahoken, and Laurent Beauguitte. “Leon Festinger, 1949, The Analysis of Sociograms Using Matrix Algebra. Version bilingue et commentée”. In: (2023).
- [FK12] Babak Farzad and Majid Karimi. “Square-Root Finding Problem In Graphs, A Complete Dichotomy Theorem”. In: (2012).
- [Fle74] Herbert Fleischner. “The square of every nonseparable graph is Hamiltonian”. In: *Journal of Combinatorial Theory, Series B* 16.1 (1974), pp. 29–34.
- [Gel70] Dennis P. Geller. “The Square Root of a Digraph”. In: *Journal of Combinatorial Theory* 9.1 (1970), pp. 65–68.
- [HKT67] Frank Harary, Richard M. Karp, and William T. Tutte. “A criterion for planarity of the square of a graph”. In: *Journal of Combinatorial Theory* 2 (1967), pp. 406–410.
- [Kön36] Dénes König. *Theorie der endlichen und unendlichen Graphen*. Akademische Verlagsgesellschaft, 1936.
- [Lau06] Lap Chi Lau. “Bipartite Roots of Graphs”. In: *ACM Transactions on Algorithms* 2.2 (2006), pp. 178–208.
- [LC04] Lap Chi Lau and Derek G. Corneil. “Recognizing Powers of Proper Interval, Split, and Chordal Graphs”. In: *SIAM Journal on Discrete Mathematics* 18.1 (2004), pp. 83–102.
- [LT07] Van Bang Le and Nguyen Ngoc Tuy. “The square of a block graph”. In: *Discrete Mathematics* 307.21 (2007), pp. 2706–2715.
- [MS94] Rajeev Motwani and Madhu Sudan. “Computing roots of graphs is hard”. In: *Discrete Applied Mathematics* 54.1 (1994), pp. 81–88.
- [Muk67] A. Mukhopadhyay. “The Square Root of a Graph”. In: *Journal of Combinatorial Theory* 2 (1967), pp. 290–295.

- [PA23] Or Peres and C. Avin. “Distributed Demand-aware Network Design using Bounded Square Root of Graphs”. In: *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications* (2023), pp. 1–10.
- [RH60] Ian C. Ross and Frank Harary. “The square of a tree”. In: *The Bell System Technical Journal* 39.3 (1960), pp. 641–647.